



VYSOKÁ ŠKOLA TECHNICKÁ V BRNĚ

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

APLIKACE PROCEDURY EXPECTIMINIMAX NA HRU VRHCÁBY

APLIKACE PROCEDURY EXPECTIMINIMAX NA HRU VRHCÁBY

BAKALÁŘSKÁ PRÁCE

BAKALÁŘSKÁ PRÁCE

AUTOR

AUTOR PRÁCE

NESTOR BARANIUK

VEDOUCÍ

VEDOUCÍ PRÁCE

doc. Ing. FRANTIŠEK ZBOŘIL, CSc.

BRNO 2025

Zadání bakalářské práce



162826

Institut: Katedra inteligentních systémů (DITS)
Student: **Baraniuk Nestor**
Studijní program: Informační technologie
Název: **Aplikace postupu ExpectiMiniMax na backgammon.**
Kategorie: Umělá inteligence
Akademický rok: 2024/25

Zadání:

1. Prostudujte pravidla hry Backgammon a postup ExpectiMiniMax.
2. Navrhněte program, který bude hrát tuto hru pomocí daného postupu.
3. Navrhněte funkce pro vyhodnocování herních stavů.
4. Vyzkoušejte navržené vyhodnocovací funkce.
5. Vyhodnoťte dosažené výsledky.

Literatura:

- Russel, S., Norvig, P.: Artificial Intelligence, A modern Approach, třetí vydání, Prentice Hall, 2010
- Simborg, P.: The Rules of Backgammon, www.WycliffeBrothers.com, 2011
- Melko E., Nagy B.: Optimal strategy in games with chance nodes, Acta Cybernetica 18, 2007

Požadavky na semestrální obhajobu:

První dva body zadání.

Podrobné formální požadavky naleznete na <https://www.fit.vut.cz/study/theses/>

Školitel: **Zbořil František, doc. Ing., CSc.**

Vedoucí katedry: Kočí Radek, Ing., Ph.D.

Zahájení práce: 1.11.2024

Termín pro podání přihlášek: 14. 5. 2025

Datum schválení: 16. 4. 2025

Abstrakt

Tato práce se zabývá implementací algoritmu ExpectiMiniMax v backgammonu, přičemž klade zvláštní důraz na vyhodnocení a upřesnění faktorů, které ovlivňují výsledky hry. Byl vyvinut systém umělé inteligence (AI) založený na algoritmu ExpectiMiniMax, který simuluje partie backgammonu mezi AI soupeři. Byly provedeny simulace zahrnující stovky partií mezi AI hráči s cílem analyzovat a kvantifikovat vliv různých strategických a pozičních faktorů v rámci hry. Výsledkem je identifikace kritických faktorů ovlivňujících hru, což umožní zlepšit schopnost AI efektivně vyhodnocovat pozice na hrací desce.

Abstrakt

Tato práce se zabývá implementací algoritmu ExpectiMiniMax ve hře Backgammon, zejména s důrazem na vyhodnocování a zdokonalování faktorů, které ovlivňují výsledky hry. Je vyvinut systém umělé inteligence založený na algoritmu ExpectiMiniMax, který simuluje hry Backgammon mezi soupeři s umělou inteligencí. Simulace, zahrnující stovky zápasů mezi hráči s umělou inteligencí, byly provedeny za účelem analýzy a kvantifikace vlivu různých strategických a pozičních faktorů ve hře. Díky tomu byly identifikovány kritické faktory ovlivňující hru, které zlepšují schopnost AI efektivně vyhodnocovat pozice na desce.

Klíčová slova

expectiminimax, alpha-beta pruning, stochastické hry, hodnocení, heuristika, backgammon, strategie

Klíčová slova

expectiminimax, alfa-beta, stochastické hry, hodnocení, heuristika, backgammon, strategie

Odkaz

BARANIUK, Nestor. *Aplikace procedury ExpectiMiniMax na backgammon*. Brno, 2025. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Školitel doc. Ing. František Zbořil, CSc.

Aplikace postupu ExpectiMiniMax na backgammon

Prohlášení

Prohlašuji, že tato bakalářská práce byla vypracována jako původní práce autora pod vedením doc. Ing. Františka Zbořila, CSc. Uvedl jsem všechny literární zdroje, publikace a další zdroje použité při vypracování této práce.

.....
Nestor Baraniuk
9. května
2025

Poděkování

Děkuji doc. Ing. Františku Zbořilovi, CSc. za podporu a pomoc při práci na této práci.

Obsah

1	Úvod	3
2	Backgammon	4
3	Algoritmy AI	6
3.1	Algoritmy pro deskové hry	6
3.2	Algoritmy pro rozhodování	7
3.2.1	MiniMax	7
3.2.2	ExpectiMiniMax	9
4	Implementace backgammonu a ExpectiMiniMax	12
4.1	Základní herní logika	12
4.1.1	Datové struktury	12
4.1.2	Průběh hry	13
4.2	Uživatelské rozhraní	14
4.3	Umělá inteligence	15
4.3.1	Architektura AI Player	15
4.3.2	Implementace ExpectiMiniMax	16
5	Optimalizace ExpectiMiniMax	22
5.1	Časová složitost a potřeba optimalizace	22
5.2	Prořezávání vyhledávacího stromu s tříděním stavů	23
5.2.1	Třídění tahů	24
5.3	Optimalizace pro hodnotící funkci	24
5.4	Paralelní zpracování výsledků hodu kostkou	26
5.5	Selektivní prohlubování	27
6	Hodnotící funkce, faktory ovlivňující hru	29
6.1	Hodnotící funkce	29
6.2	Faktory	31
6.3	Testy	35
6.3.1	Relevance faktorů	36
6.3.2	Zápasy proti lidským hráčům	38
7	Závěr	40
	Bibliografie	41
	Návod k použití	42
B	Očekávaný počet získaných bodů za tah	45

Seznam obrázků

2.1	Počáteční nastavení	5
3.1	Příklad herního stromu MiniMax. Trojúhelníky představují hráče MAX a MIN. Listové uzly zobrazují hodnoty užitku.	8
3.2	Příklad herního stromu ExpectiMiniMax. Obsahuje hráče MAX a MIN jako trojúhelníkové uzly a uzly náhody jako kruhy.	10
4.1	Uživatelské rozhraní Backgammonu. Počáteční nastavení.....	14
4.2	Zvýraznění bodů	15
4.3	Permutace	17
5.1	Příklad alfa-beta prořezávání	23
5.2	Struktura úkolů	26
6.1	Základní hodnotící funkce a hlavní faktory	31
6.2	Problém s hodnocením	32
6.3	Výherní poměr podle faktoru.....	37
6.4	Kombinace faktorů vs. základní hodnocení.....	38

Kapitola 1

Úvod

Umělá inteligence (AI) je široce využívána v deskových hrách. Jedná se o efektivní řešení, ale úkol se stává velmi komplikovaným, když do hry vstupuje nejistota. Vytváří to mnoho možných variant vývoje hry a předvídat různé možnosti se stává náročným. Jednou z takových her je backgammon.

V dnešní době můžeme využít AI k hraní her s nejistotou a k jejich lepšímu pochopení. Počítače dokážou relativně rychle simulovat tisíce her, což nám pomáhá objevovat nové strategie a rozhodnutí, která zlepšují hratelnost. Tato práce se zabývá výukou počítače, jak inteligentně hrát backgammon pomocí metod, které zvládají nejistotu, podobně jako se lidští hráči přizpůsobují měnícím se hodům kostkami. Téma je důležité, protože pochopení toho, jak zlepšit herní rozhodnutí počítače, nám pomáhá poznat rozhodování v podmínkách nejistoty. I dnes, kdy se AI výrazně zlepšila a v mnoha hrách snadno překonává lidské šampiony, nejsou tyto hry vyřešeny a neustále hledáme optimálnější řešení. Vždy existuje prostor pro růst a zvýšení efektivity.

Hlavním cílem této práce je implementace konkrétního algoritmu umělé inteligence známého jako ExpectiMiniMax pro hraní backgammonu. Simulací mnoha zápasů mezi umělými inteligencemi práce zkoumá, které faktory skutečně ovlivňují kvalitu hry, a tyto poznatky využívá ke zlepšení způsobu, jakým počítač vyhodnocuje různé herní pozice.

Kapitola 2 se zabývá základy backgammonu, jeho pravidly a tím, jak se tradičně hraje. Kapitola 3 vysvětluje algoritmy umělé inteligence a poskytuje informace o tom, jak počítače činí rozhodnutí. Kapitola 4 popisuje implementaci samotné hry backgammon v jazyce C# a algoritmu ExpectiMiniMax. Kapitola 5 se věnuje tomu, jak tuto implementaci optimalizovat pro lepší výkon. Kapitola 6 zkoumá faktory ovlivňující výsledky hry a způsoby, jak zlepšit hodnocení herních stavů. V této kapitole byl experimentálně stanoven vliv těchto faktorů na hru.

Kapitola 2

Backgammon

Backgammon je hra pro dva hráče, ve které každý účastník pohybuje svými kameny podle výsledků hodu kostkami. Jedná se o jednu z nejstarších známých deskových her, která kombinuje strategii a pravděpodobnost. Tato část shrnuje pravidla hry, která tvoří základ pro implementaci kódu hry a algoritmu ExpectiMiniMax. Podrobnosti o tomto algoritmu budou uvedeny v následujících kapitolách.

Pravidla backgammonu

Pravidla hry a terminologie byly převzaty z webové stránky Backgammon Galore [2].

- Cílem **backgammonu** je, aby hráč přesunul všechny své **kameny** do své **domácí desky** a poté **je vyřadil**. Hru vyhrává první hráč, který odstraní všechny své kameny z hrací desky.
- Backgammonová deska se skládá z **24 bodů**. Každý hráč má **domácí desku** a **vnější desku**, oddělené centrální oblastí zvanou **bar**. Každý hráč se pohybuje opačným směrem: jeden ve směru hodinových ručiček, druhý proti směru hodinových ručiček.
- Každý hráč začíná s **15 kameny** umístěnými na konkrétních polích. Standardní rozložení zahrnuje:
 - Dvě kameny na bodu **24**
 - Pět na **13. bodu**
 - Tři na bodě **8**
 - pět na bodu **6**

Toto pevné rozložení definuje **počáteční stav hry** (obrázek 2.1).

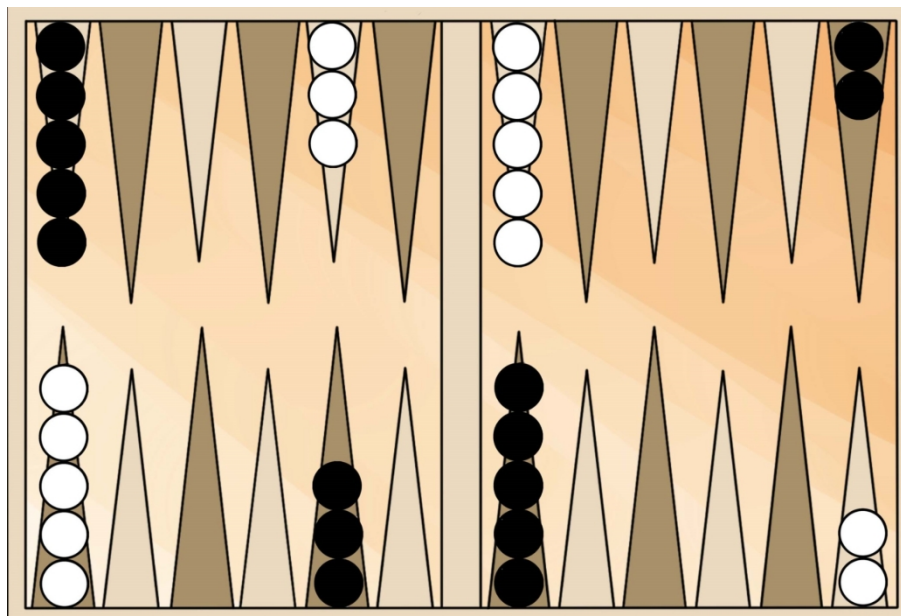
- Na začátku hry hodí každý hráč jednou kostkou. Hráč s vyšší hodnotou hází jako první. Pokud hráči hodí stejnou hodnotu, hází znovu, dokud není určen vítěz. V dalších tazích hráči hodí **dvěma šestistrannými kostkami** a pohybují kameny podle hodu. Kámen se může přesunout na:
 - Neobsazené pole
 - Bod obsazený kameny hráče
 - Bod s jedinou kamenem soupeře (**blot**)

- Bod s **dvěma nebo více kameny soupeře** je **zablokován** a nelze na něj přesunout. Hráči musí volit platné cíle a **nemohou pasovat**, pokud nejsou k dispozici žádné povolené tahy.
- Každá kostka představuje jeden tah. Hráč může:
 - Pohnout **jedním kamenem** pomocí obou hodnot
 - Pohnout **dvěma kameny** samostatně

Hráč musí použít co nejvíce hodnot kostek a nemůže vynechat tah, pokud neexistují žádné povolené tahy.

Pokud padne **dvojice** (např. 5–5), musí hráč provést 4 tahy. Pokud hráč přesune jednu kamenku vícekrát s využitím obou hodnot kostek, musí být každý tah sám o sobě povolený a kamenka musí při každém mezikroku přistát na volném nebo platném bodu; blokové body nelze projít.

- Přistání na bodě s **jedinou soupeřovou kamenou** ji **srazí** a pošle na **bar**. Kámen na baru musí být **znovu uveden do domácího pole** soupeře. Znovuvvedení vyžaduje **volný bod** odpovídající hodnotě jedné z kostek.
- Jakmile jsou všechny kameny v domácí desce hráče, může začít s **vyřazováním**. Hod kostkou musí odpovídat vzdálenosti kamene od okraje (bod 1 pro bílé nebo bod 24 pro černé). Pokud není možné dosáhnout přesné shody, může hráč vyřadit **nejvzdálenější kámen v dosahu**.
- Pokud není možný žádný povolený tah, musí hráč **vynechat svůj tah**. K tomu může dojít, pokud:
 - Všechny cílové body jsou **blokovány**
 - Je kámen na **baru** bez platného vstupního bodu



Obrázek 2.1: Počáteční rozestavení

Kapitola 3

Algoritmy AI

Tato kapitola se zaměří na popis využití AI v problémech soupeřivého hledání známých jako hry. Využití je široké a zahrnuje různé přístupy k vytváření prostředí s více agenty. Jejich výběr závisí hlavně na složitosti hry, náhodnosti a úrovni dostupných informací.

3.1 Algoritmy pro deskové hry

- **Rozhodovací algoritmy založené na hledání** simulují racionální uvažování tím, že prozkoumávají možné budoucí stavy hry. Metody jako MiniMax a ExpectiMiniMax jsou široce používány v šachu a backgammonu.
- **Reinforcement Learning (RL)** umožňuje agentovi AI učit se strategie metodou pokusů a omylů, přičemž za úspěšné akce dostává odměny. TD-Gammon je dokonalým příkladem využití reinforcement learningu [12].
- **Metody Monte Carlo** používají náhodné simulace (rollouty) k vyhodnocení tahů a odhadu výsledků. Například Monte Carlo Tree Search (MCTS) bylo použito k vývoji AlphaGo [3]. V kombinaci s neuronovými sítěmi pomohla tato metoda AI v roce 2015 poprvé porazit lidského hráče ve hře Go a poté v roce 2017 porazit mistra v Go.
- **Neuronové sítě a hluboké učení** jsou široce používané techniky. Umožňují AI učit se vzorce a hodnotící funkce přímo z herních dat. Například AlphaZero a MuZero kombinovaly hluboké neuronové sítě s MCTS k hraní šachů, šogi a Go [5].
- **Evoluční algoritmy** jsou inspirovány přirozenou evolucí, takže tyto metody vyvíjejí strategie nebo hodnotící funkce v průběhu generací. Takové genetické algoritmy byly použity k trénování AI v Othellu [7].
- **Učení založené na případech nebo napodobování** zahrnuje pozorování a napodobování tahů expertů z předchozích her. Příkladem je trénování AI na velkých šachových databázích za účelem replikace herních vzorců velmistrů. Tato metoda se běžně používá k zahájení prvních několika tahů nebo k ukončení šachových partií s předem určenými výsledky. Například je vyřešena každá šachová partie, ve které je na šachovnici maximálně sedm figur [9].

Následující části se zaměří na **algoritmy pro rozhodování**, jako je **algoritmus MiniMax** a jeho stochastické rozšíření **ExpectiMiniMax**.

3.2 Algoritmy pro rozhodování

Následující výklad algoritmů vychází z [11, s. 161–180].

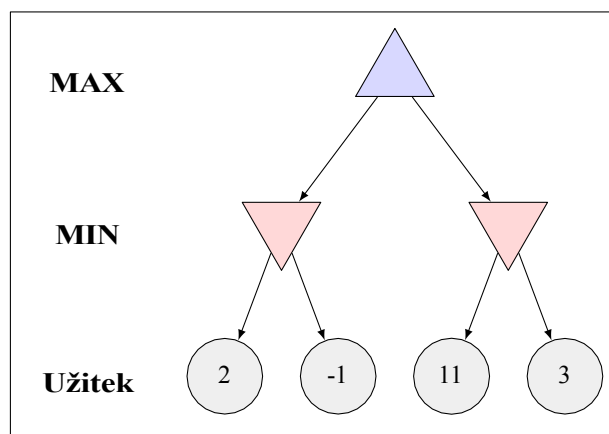
Algoritmy pro rozhodování se běžně používají pro **hry** dvou hráčů s **nulovým součtem**, jako jsou **backgammon**, šachy, dáma, šogi a go. Všechny tyto hry, s výjimkou backgammonu, jsou také hrami s **dokonalou informovaností**. To znamená, že jsou **deterministické** a **plně pozorovatelné** (oba hráči vidí aktuální pozici toho druhého a všechny možné tahy). V těchto hrách se hráči střídají. Jsou-li hráči reprezentováni jako **agenti**, je jejich užitková hodnota (výsledek **hodnotící funkce**, která vrací číselnou hodnotu stavu hrací desky) buď stejná, nebo opačná. Algoritmy pro rozhodování vycházejí z požadavku hry učinit určitá rozhodnutí, i když nelze najít optimální rozhodnutí (nejlepší tah).

3.2.1 MiniMax

Jedním z nejčastěji používaných algoritmů pro rozhodování je **MiniMax**. Jeho princip je velmi jednoduchý: cílem každého hráče je vyhrát, což ho vede k provádění nejlepších možných tahů směřujících k tomuto cíli. V hrách s nulovým součtem zisk jednoho hráče přímo odpovídá ztrátě druhého. Úkol je triviální, pokud je hra dostatečně jednoduchá na to, aby bylo možné získat všechny stavy. V takovém případě hráč jednoduše vybere posloupnost tahů, která ho dovede k vítězství. Ve většině her je však nemožné vypočítat všechny potenciální výsledky. Pro analýzu a hledání optimálních tahů v takových hrách se využívá **herní strom**, který se skládá ze 3 částí:

- Počáteční stav hry jako kořen stromu.
- Potenciální stavy hry jako uzly.
- Akce hráče (tahy) jako hrany.

Jak hra postupuje, strom se rozšiřuje a zobrazuje všechny možné sekvence tahů. Na listech tohoto stromu — kde je vyhledávání zastaveno z důvodu omezení hloubky nebo času — se používá **heuristická hodnotící funkce** k určení, jak příznivý je výsledný stav hry pro hráče. Na rozdíl od užitkové funkce hodnotí heuristická funkce mezistavy, které nejsou konečné, a přeměňuje je na listové uzly stromu. Není dokonalá, protože dokáže pouze odhadnout hodnotu užitkové funkce, takže v tomto kontextu odráží spíše pravděpodobnost výhry než samotné vítězství. Algoritmus MiniMax zohledňuje dva hráče: **MAX**, který chce maximalizovat hodnotu užitku, a **MIN**, který ji chce minimalizovat. Tito hráči se střídají v pohybu po stromu. MAX předpokládá, že MIN vždy vybere tah, který je pro MAX nejméně výhodný, a naopak. Při hodnocení tahů pomocí heuristické funkce si MAX vždy vybere tah s nejvyšší hodnotou a MIN s nejnižší hodnotou. Rekursivním vyhodnocováním stromu umožňuje MiniMax AI vybrat tah, který zajistí nejlepší možný výsledek, za předpokladu, že oba hráči hrají optimálně. V části 3.1 je uveden příklad takového stromu. Kořenovým uzlem je MAX; ten vybírá nejvyšší hodnotu ze svých dvou podřízených uzlů — MIN. Současně je hodnotou MIN nejnižší hodnota jejich podřízeného uzlu, což je v tomto případě výsledek užitkové funkce.



Obrázek 3.1: Příklad herního stromu MiniMax. Trojúhelníky představují hráče MAX a MIN. Listové uzly zobrazují hodnoty užitku.

V **tahových hrách** se akce jednoho hráče nazývá **tah**. Jelikož se hráči střídají, jeden kompletní tah ve hře se skládá ze dvou tahů – jednoho od každého hráče.

Hloubka prohledávání v tomto kontextu je vzdálenost v tazích mezi kořenem stromu (aktuálním stavem) a nejvzdálenějším **listovým uzlem**. Některé listové uzly stromu představují **konečné stavy**. Tyto herní stavy určují, že hra skončila (je vítěz nebo remíza) nebo že z tohoto stavu neexistují žádné povolené tahy. Pevný počet tahů obvykle omezuje hloubku, aby se šetřily výpočetní zdroje, protože počet potenciálních stavů hry roste **exponenciálně** s rostoucí hloubkou.

Definice algoritmu

Algoritmus MiniMax pracuje **rekurzivně** tím, že prozkoumává možné tahy do určité hloubky. Za předpokladu, že herní strom je již vygenerován, algoritmus MiniMax je následující:

1. Přijme **uzel** aktuálního **herního stavu**, zbývající hloubku prohledávání a příznak označující, zda aktuální hráč maximalizuje nebo minimalizuje.
2. Pokud je hloubka nulová nebo je uzel koncovým stavem, vrátí hodnotu aktuálního stavu pomocí **užtkové (hodnotící) funkce**.
3. **Hráč maximalizující:** Pokud se aktuální hráč snaží maximalizovat skóre:
 - Pro každý poduzel (představující možný tah) rekurzivně voláme MiniMax s hloubkou sniženou o jednu a rolí přepnutou na minimalizaci.
 - Sledujte maximální hodnotu vrácenou z rekurzivních volání.
 - Vraťte nejvyšší získanou hodnotu.
4. **Hráč s cílem minimalizovat skóre:** Pokud se aktuální hráč snaží minimalizovat skóre:
 - Pro každý poduzel rekurzivně zavolejte MiniMax se sniženou hloubkou a s rolí přepnutou na maximalizaci.
 - Sledujte vrácenou minimální hodnotu.
 - Vraťte nejnižší získanou hodnotu.

Algoritmus 1 Algoritmus MiniMax

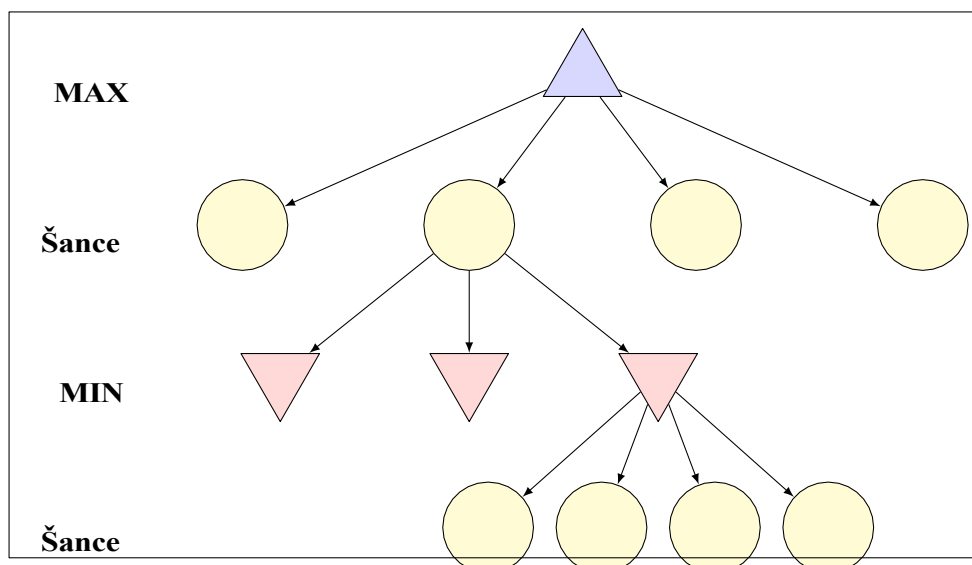
```
1: funkce MINIMAX(node , depth, MaximizingP layer )
2:   pokud depth = 0 nebo ISTERMAL(node ), pak
3:     vrátit VYHODNOTIT(node )
4:   konec
5:   if MaximizingP layer then
6:     value  $\leftarrow -\infty$ 
7:     pro každé dítě node proved'
8:       value  $\leftarrow \max(\text{value}, \text{MINIMAX}(\text{child}, \text{depth} - 1, \text{False}))$ 
9:     konec pro
10:    návrat value
11:  jinak
12:    value  $\leftarrow +\infty$ 
13:    pro každé dítě node proved'
14:      value  $\leftarrow \min(\text{value}, \text{MINIMAX}(\text{child}, \text{depth} - 1, \text{True}))$ 
15:    konec pro
16:    vrátit value
17:  konec if
18: konec funkce
```

Lze jej také znázornit jako pseudokód (algoritmus 1). Algoritmus MiniMax má jednu významnou nevýhodu: má **exponenciální časovou složitost** $O(b^m)$, kde b je **faktor větvení**. Tento faktor představuje počet potomků daného uzlu m představuje hloubku. Existují však různé metody a **optimalizační techniky**, jako je **alfa-beta prořezávání**, které slouží ke snížení počtu uzlů, které je třeba vyhodnotit. Podrobnější popis těchto optimalizací je uveden v kapitole 5.

3.2.2 ExpectiMiniMax

Algoritmus MiniMax je vynikající pro hry s **dokonalou informacemi**, kde jsou všechny možné tahy předem určeny. Hra však může být ovlivněna vnějšími faktory, které přidávají **nejistotu**. Takové hry se nazývají **stochastické hry** a **backgammon** je jednou z nich. V této hře závisí dostupné tahy na **hodu kostkami**, což vytváří mnohem širší škálu možných herních stavů. V hrách, jako je backgammon, není možné předpovědět sadu povolených tahů soupeře, takže je nutné zvážit všechny možné kombinace jeho tahů. Herní strom by měl být také rozšířen o další **uzly náhody**. Tyto uzly jsou nezbytné pro znázornění nejistoty a jsou umístěny mezi uzly MAX a MIN jako jejich potomci. Nejistota je odvozena z hodu kostkami. Existuje 21 jedinečných výsledků hodu dvěma kostkami, které jsou reprezentovány jedním z 21 podřízených uzlů. Hrany spojující uzly náhody s jejich podřízenými uzly označují **pravděpodobnost** každého konkrétního hodu kostkou. Pravděpodobnost je $\frac{1}{36}$ pro všechny **dvojice** (například 5–5 nebo 2–2), zatímco je $\frac{1}{18}$ pro všechny

ostatních hodů. Je to proto, že hody jako 1–3 a 3–1 jsou ekvivalentní, každý s pravděpodobností $\frac{1}{36}$, což vede ke kombinované pravděpodobnosti $\frac{1}{18}$.



Obrázek 3.2: Příklad herního stromu ExpectiMiniMax. Obsahuje hráče MAX a MIN jako trojúhelníkové uzly a uzly šance jako kruhy.

Při srovnání časové složitosti tohoto algoritmu s algoritmem MiniMax je zřejmé, že ExpectiMiniMax pro backgammon lze chápat jako rozšíření MiniMaxu. Místo toho, aby se rekurzivně volal pouze jednou, volá se 21krát – jednou pro každý jedinečný hod kostkou. Tento vzorec pokračuje na každé následující úrovni hloubky, což vede k multiplikátoru 21^m , kde m představuje hloubku. Použitím tohoto faktoru na časovou složitost MiniMaxu, která je $O(b^m)$, lze odvodit složitost $O(21^m b^m)$. Pro zobecnění vzorce, pokud označíme počet různých hodů jako n , je složitost $O(n^m b^m)$.

Počet herních stavů, které je třeba vyhodnotit, lze odhadnout pouze přibližně v každé hloubce. Testy popsané v kapitole 6 ukázaly, že průměrný faktor rozvětvení se pohybuje mezi 15 a 23, přičemž na tuto hodnotu má významný vliv konkrétní hra. V důsledku toho je obtížné určit přesný průměr, lze jej však odhadnout na přibližně 19. V případech, kdy padne dvojice, může tento faktor vzrůst na desítky či dokonce stovky. I při faktoru rozvětvení 15 by prohledávání ve dvou úrovních zahrnovalo $(2^{15})^2 = 99\,225$ různých herních stavů, zatímco ve třech úrovních se tento počet zvyšuje na $(2^{15})^3 = 3,13 \times 10^7$ stavů. Po optimalizaci lze čas potřebný pro výpočty zkrátit až stokrát a může se snížit i míra růstu. Stále však zůstává exponenciální, což značně ztěžuje výpočet potenciálních herních stavů v hloubce čtyř tahů a více.

Kód algoritmu ExpectiMiniMax je podobný algoritmu MiniMax, přičemž klíčovým rozdílem je přidání náhodných uzlů a způsob výpočtu hodnot v těchto uzlech. Níže je uveden vzorový pseudokód tohoto algoritmu.

Algoritmus 2 Algoritmus ExpectiMiniMax

```
1: funkce EXPECTIMINIMAX(node )
2:   if ISTERMIAL(node ) then
3:     vrátit EVALUATE(node )
4:   konec
5:   if ISMAXNODE(node ) then
6:     value  $\leftarrow -\infty$ 
7:     pro každé child znode proved'
8:       value  $\leftarrow \max(\textit{value}, \text{EXPECTIMINIMAX}(\textit{child}))$ 
9:     konec pro
10:    návrat value
11:  jinak pokud ISMINNODE(node ) pak
12:    value  $\leftarrow +\infty$ 
13:    pro každé child znode proved'
14:      value  $\leftarrow \min(\textit{value}, \text{EXPECTIMINIMAX}(\textit{child}))$ 
15:    konec pro
16:    návrat value
17:  jinak pokud ISCHANCENODE(node ) pak
18:    value  $\leftarrow 0$ 
19:    pro každé child znode proved'
20:      prob  $\leftarrow \text{GETPROBABILITY}(\textit{node} , \textit{child})$ 
21:      value  $\leftarrow \textit{value} + \textit{prob} \times \text{EXPECTIMINIMAX}(\textit{child})$ 
22:    konec pro
23:    návrat value
24:  konec if
25: konec funkce
```

Jelikož je nemožné předpovědět výsledek hodu kostkou, je nezbytné vyhodnotit každou možnost spolu s její pravděpodobností. V tomto případě je třeba získat průměrný výsledek všech možných výsledků z náhodného uzlu, vypočítaný jako součet těchto hodnot vynásobený jejich pravděpodobnostmi. Tento výpočet dává výsledek známý jako očekávaná hodnota. Tímto způsobem se zohlední každý potenciální scénář. Očekávanou hodnotu lze formálně definovat jako:

$$\text{Hodnota}(n) = \sum_{i=1}^k p_i \cdot f(c_i)$$

- $\text{Hodnota}(n)$: očekávaná hodnota uzlu pravděpodobnostin .
- k : celkový počet potomků uzlu n .
- c_i : i -tý potomkový uzel uzlu n .
- p_i : pravděpodobnost přechodu z uzlu n na jeho i -tého potomka c_i .
- $f(c_i)$: vyhodnocená hodnota poduzlu c_i , která může být vypočítána rekurzivně v závislosti na tom, zda je c_i uzlem MAX, MIN nebo koncovým uzlem.

Kapitola 4

Implementace Backgammonu a ExpectiMiniMax

Hra Backgammon byla implementována v programovacím jazyce C# s použitím verze 12.0 a frameworku .NET 8.0. Aplikace se skládá ze dvou hlavních částí: grafického uživatelského rozhraní a jádra herní logiky. Uživatelské rozhraní využívá Windows Presentation Foundation (WPF), což poskytuje flexibilní a interaktivní desktopové prostředí. Herní logika je zapouzdřena v samostatné knihovně tříd s názvem *Core*. Tato knihovna zpracovává pravidla hry, spravuje stav hry a provádí platné tahy. Obsahuje také implementaci algoritmu ExpectiMiniMax používaného hráčem AI. Aplikace WPF používá knihovnu *Core* k synchronizaci vizuální hrací desky s herní logikou a ke zpracování interakcí uživatele nebo AI.

4.1 Základní herní logika

4.1.1 Datové struktury

Jádro hry je postaveno na několika klíčových datových strukturách reprezentujících stav hrací desky, hráče a kameny. Tyto struktury jsou implementovány jako třídy C# uvnitř knihovny **Core**. Každá entita je navržena tak, aby co nejvěrněji odrážela základní komponenty hry Backgammon a její logiku.

Třída *Checker* představuje jednu hrací figurku. Každá figurka má pozici a barvu (bílou nebo černou), která se používá k určení vlastnictví. Třída *Point* modeluje jedno z 24 míst na backgammonové desce. Obsahuje seznam kamenů, poskytuje metody pro jejich přidávání nebo odebírání a kontroluje vlastnictví nebo to, zda obsahuje blot.

Třída *Player* sleduje barvu každého hráče a kameny, které aktuálně vlastní. Je také zodpovědná za ukládání kamenů přesunutých na bar nebo vyřazených. Třída *GameBoard* kombinuje všech 24 bodů do jedné struktury a spravuje logiku lajny, vyřazování kamenů a jejich pohyby.

Třída *Game* propojuje hrací desku, hráče a kostky, aby spravovala celkový stav hry. Sleduje také, na kterém hráči je tah a zda hra skončila. Tato třída je ústředním kontrolním bodem pro ověřování a provádění tahů, provádění tahů a určování výsledku hry.

Program podporuje klonování herních stavů. To se provádí hlubokým zkopírováním celé desky a jejích kamenů do nového objektu *GameBoard*. Tato funkce je nezbytná pro AI

simulovat budoucí tahy, aniž by to ovlivnilo hru. Každá klonovaná deska si zachovává plně rozložení a vlastnictví kamenů a umožňuje bezpečné vyhodnocení hypotetických výsledků.

4.1.2 Průběh hry

Průběh je řízen třídou `Game` a podporován pomocnými funkcemi a dalšími základními třídami.

Na začátku nové hry je hrací deska inicializována s kameny umístěnými na jejich standardních pozicích. Toto nastavení je zpracováno v konstruktoru `Game` prostřednictvím metody `InitializeCheckers`. Každému hráči je přiděleno 15 kamenů, které jsou rozloženy na konkrétních bodech na hrací desce.

Tah začíná hodem kostkou pomocí metody `RollDice` třídy `Dice`. Tato třída zpracovává běžné a dvojité hody, přičemž dvojité hody umožňují až čtyři tahy. Aktuální hráč pak použije hodnocené hodnoty k provedení platných tahů vstupem z bar, vyřazením nebo přesunem kamenů po hrací desce. Pokud má hráč kameny na baru, musí je nejprve vyřadit, než provede další tahy.

Statické metody `IsValidMove`, `IsValidBarMove` a `CanBearOffFromIndex` vynucují pravidla hry týkající se povolených tahů, směru pohybu, návratu z bar a podmínek vyřazení kamenů. Jedná se o pomocné metody pro ověřování platnosti tahů. Kromě toho může hra pomocí těchto metod vrátit seznam možných cílových bodů pro daný výchozí bod.

Během tahu hráče herní engine pomocí metody `HasAvailableMoves` zkontroluje, zda může hráč provést nějaký platný tah. Pokud není možný žádný tah, tah se automaticky přenechá soupeři. V opačném případě hráč provede tahy pomocí metod `MakeMove` nebo `BearOffChecker`.

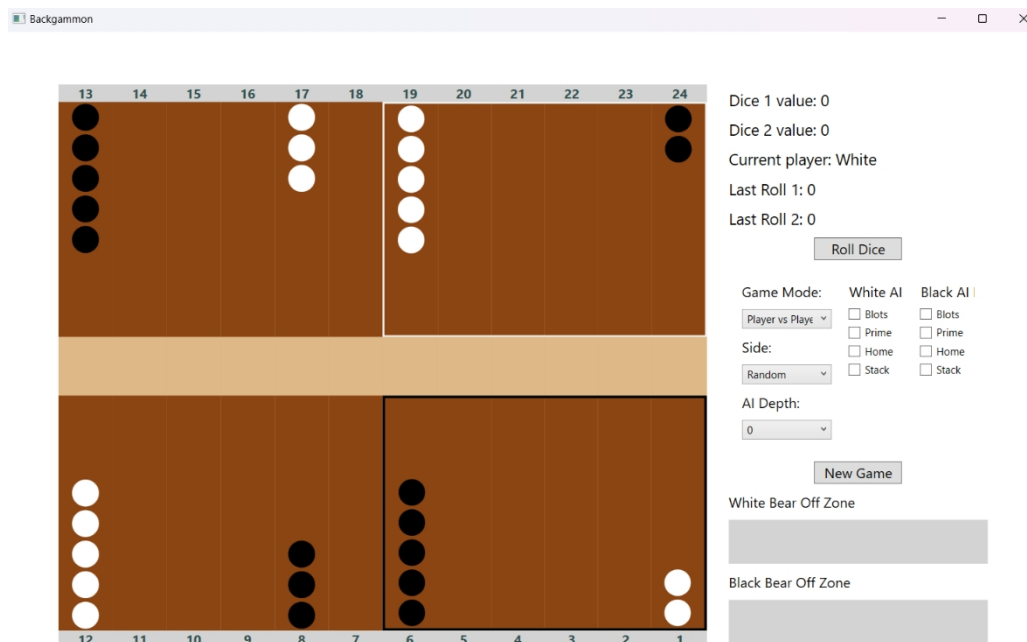
Poté, co jsou provedeny nebo přeskočeny všechny platné tahy, hra zkontroluje podmínky vítězství pomocí metody `CheckForWinner`. Pokud byly všechny kameny hráče úspěšně vyřazeny a žádné nezůstaly na baru nebo na desce, hra končí a je vyhlášen vítěz.

Pokud hra neskončila, metoda `EndTurn` vynuluje kostky a přepne aktuálního hráče pomocí metody `SwitchTurn`. Nový tah začíná okamžitě a cyklus pokračuje, dokud jeden z hráčů nesplní podmínku vítězství. Lze to znázornit jako jednoduchý algoritmus:

1. Spustíte novou hru.
2. Opakujte následující kroky, dokud hra neskončí:
 - (a) Hod'te kostkami.
 - (b) Zkontrolujte, zda má aktuální hráč nějaké platné tahy.
 - (c) Pokud existují platné tahy:
 - Proved'te tahy na základě hodnot na kostkách.
 - (d) Pokud nejsou k dispozici žádné platné tahy:
 - Předávejte tah soupeři.
 - (e) Zkontrolujte, zda aktuální hráč vyhrál.
 - (f) Pokud není nalezen vítěz, změňte aktivního hráče.
3. Ukončete hru a vyhlášte vítěze.

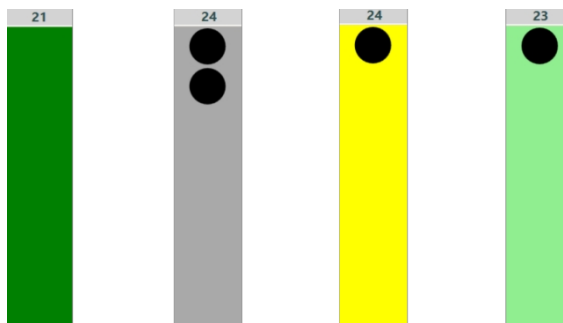
4.2 Uživatelské rozhraní

Při navrhování uživatelského rozhraní byly hlavními prioritami jednoduchost a přehlednost. Cílem nebylo zaměřit se na grafickou estetiku, ale vytvořit funkční a srozumitelné rozvržení, které je úzce propojeno s herní logikou. Rozhraní je vytvořeno pomocí Windows Presentation Foundation (WPF) a skládá se z jediného okna, které obsahuje jak herní desku, tak ovládací prvky. Toto jednotné rozvržení umožňuje hráči přímo interagovat s hrou a sledovat všechny relevantní informace na jednom místě. Hrací deska vizuálně znázorňuje všech 24 bodů, kameny a oblasti barů, zatímco ovládací prvky umožňují akce, jako je házení kostkami, zahájení nové hry a zobrazení aktuálního hráče.



Obrázek 4.1: Uživatelské rozhraní hry Backgammon. Počáteční nastavení

Obrázek 4.1 ukazuje na levé straně hrací desku s 24 očíslovanými body, na které se kameny umísťují a přesouvají. Střední pruh slouží jako čekací zóna pro kameny, které jsou do něj posunuty. Dvě zóny pro vyřazení kamenů se nacházejí vpravo dole a zobrazují vyřazené kameny bílého a černého hráče. Na pravé straně okna jsou k dispozici různé ovládací prvky: hodnoty hodu kostkami, indikátor aktuálního hráče, paměť posledního hodu a tlačítko „Hodit kostkami“. Rozbalovací nabídky a zaškrťovací políčka umožňují hráčům vybrat herní režim, chování AI, stranu, za kterou hrají, a hloubku prohledávání pro AI. Tlačítko „Nová hra“ rešetuje stav hry a vrátí hrací desku do počátečního nastavení. Jediným významným rozdílem je, že pruh na klasické desce ji rozděljuje svisle na polovinu, zatímco v této implementaci je vodorovný. Pro oddělení domovských bodů od vnějších bodů mají domovské body ohraničení v barvě odpovídající hráči, ke kterému patří.



Obrázek 4.2: Zvýraznění bodů

Důležitou vizuální pomůckou v uživatelském rozhraní je funkce zvýraznění tahu. Když hráč vybere kámen, bod, na kterém se nachází, se zvýrazní šedě a všechny platné cílové body se zvýrazní zeleně. To pomáhá hráči snadno identifikovat povolené tahy na základě aktuálního hodu kostkou. Dostupné cíle se vypočítají pomocí jádra a zobrazí se v uživatelském rozhraní. Zvýraznění se resetuje po dokončení nebo zrušení tahu.

Obrázek 4.2 ilustruje několik typů zvýraznění bodů používaných během tahu hráče, zobrazených zleva doprava:

1. Bod, který představuje platný cíl pro aktuálně vybraný bod na základě aktuálních nevyužitých hodnot kostek.
2. Bod aktuálně vybraný hráčem.
3. Bod, ze kterého již byla v aktuálním tahu přesunuta figurka.
4. Bod, do kterého již byla figurka přesunuta v rámci stejné sekvence tahů.

4.3 Umělá inteligence

Systém umělé inteligence (AI) je navržen tak, aby simuloval soutěživého soupeře v backgammonu. Jeho primárním cílem je vyhodnotit aktuální stav hry, předvídat budoucí možnosti a činit rozhodnutí, která maximalizují jeho šanci na výhru. AI přímo interaguje s jádrem herní logiky a řídí se stejnými pravidly jako lidský hráč, včetně náhodnosti založené na kostkách a omezení povolených tahů. Tato sekce popisuje vnitřní architekturu komponenty AI a implementaci algoritmu ExpectiMiniMax, který umožňuje inteligentní rozhodování.

4.3.1 Architektura AI hráče

AI je implementována jako specializovaná třída v rámci knihovny `Core` a je inicializována odkazem na herní desku a hráče, kterého reprezentuje. Je konstruována s ohledem na oba hráče a funguje nezávisle na uživatelském rozhraní. AI se zapojuje automaticky během svého tahu a nevyžaduje přímou interakci uživatele.

Pro rozhodování AI přijímá aktuální stav hry a hod kostkou a pomocí metody `GenerateUniqueStates` generuje všechny možné sekvence tahů, které má k dispozici. Klíčovou metodou zodpovědnou za toto je `GetBestMove`, která vrací seznam tahů představujících

optimální strategii na základě aktuálního stavu. Interně tato metoda volá algoritmus ExpectiMiniMax, aby vyhodnotila různé budoucí výsledky a vybrala cestu s nejvyšší očekávanou hodnotou.

Umělá inteligence podporuje konfigurovatelné hodnotící funkce kombinací několika bodovacích faktorů, které budou podrobně popsány v kapitole 6. Tyto faktory se při vyhodnocování herních stavů uplatňují dynamicky. Jakmile je určena nejlepší sekvence tahů, hra je provede stejným způsobem jako u lidského hráče.

4.3.2 Implementace ExpectiMiniMax

Hráč AI činí rozhodnutí simulací hry o několik tahů dopředu, přičemž zohledňuje jak deterministické akce, tak stochastické prvky dané hodem kostkou. Jeho chování je řízeno algoritmem ExpectiMiniMax, který vyhodnocuje možné sekvence tahů a vybírá tu s nejvyšší očekávanou užitečností. Celý rozhodovací proces hráče AI se skládá z následujících kroků:

1. Prozkoumat všechny povolené akce na základě aktuálních hodnot kostek.
2. Simulace všech výsledných stavů na hrací desce a jejich uložení do prohledávatelné struktury.
3. Pro každý výsledný stav simulovat potenciální reakce soupeře na základě všech možných výsledků hodu kostkou.
4. Pro každou z těchto reakcí soupeře simulovat potenciální odpovědi AI pomocí další úrovně ExpectiMiniMax.
5. Vyhodnoťte konečné stavy pomocí konfigurovatelné hodnotící funkce založené na heuristických faktorech.
6. Vyberte a vraťte počáteční sekvenci tahů, která vede k nejvyššímu očekávanému výsledku.

V následujících podkapitolách bude každá část algoritmu popsána podrobněji, počínaje vstupním bodem procesu.

GetBestMove

Metoda `GetBestMove` slouží jako vstupní bod pro rozhodovací proces AI. Je volána na začátku tahu AI a přijímá dva parametry: aktuální hodnoty kostek (již hozených) a hloubku prohledávání. Tato metoda je zodpovědná za generování a vyhodnocení všech možných výsledků, kterých může AI dosáhnout pomocí aktuálních kostek.

Prvním krokem prováděným metodou `GetBestMove` je generování všech jedinečných herních stavů, kterých

AI hráč dosáhnout, vzhledem k aktuálním hodnotám kostek. Toho je dosaženo pomocí metody `GenerateUniqueStates`, která prozkoumává různé sekvence tahů rekursivním aplikováním platných tahů a simulací jejich výsledků na klonovaných kopiích hrací desky. Pro každý jedinečný výsledný stav je zaznamenána sekvence tahů a přiřazena k hashované reprezentaci hrací desky.

Jakmile jsou shromážděny všechny dosažitelné stavy, algoritmus je seřadí tak, aby upřednostnil stavy, ve kterých má soupeř více kamenů na liště. To vychází z předpokladu, že srážení kamenů je často výhodné a může omezit reakce soupeře.

Pro každý výsledný stav metoda rekursivně volá funkci `ExpectiMiniMax`. Tato funkce zkoumá budoucí výsledky, střídá tahy AI a soupeře a zároveň zpracovává náhodnost

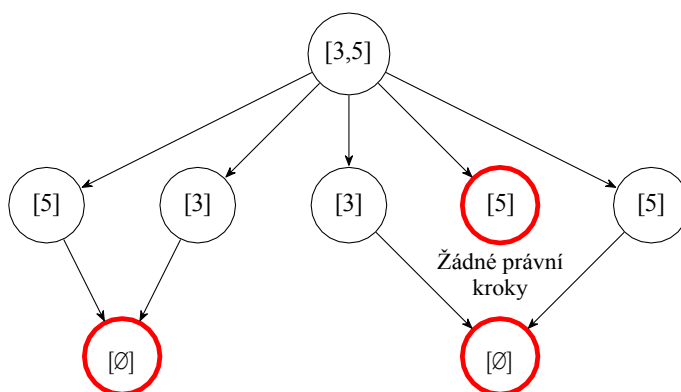
. Po vyhodnocení všech možných výsledků vrátí `GetBestMove` sekvenci tahů, která vede ke stavu s nejlepším skóre hodnocení.

Interně jsou všechny sekvence tahů uloženy ve slovníku nazvaném `stateToMoves`, který mapuje hashované stavy hrací desky na odpovídající sekvence tahů. Jakmile je nalezen nejlepší konečný stav, jsou skutečné tahy načteny pomocí `RetrieveMoveSequence` a provedeny AI v pořadí.

GenerateUniqueStates

Metoda `GenerateUniqueStates` je zodpovědná za generování všech možných stavů hrací desky, kterých může hráč AI dosáhnout z dané výchozí desky a hodnot kostek. Tato funkce je nezbytná pro rozšiřování herního stromu při každém tahu, protože prozkoumává každou jedinečnou sekvenci tahů, která je hráči k dispozici.

Metoda pracuje rekurzivně a operuje s klonovanými kopiemi aktuální hrací desky. Na každé úrovni rekurze se pokouší aplikovat každý platný tah s použitím aktuálních hodnot kostek. Pokaždé, když je simulován tah, funkce se znovu volá s výslednou hrací deskou a zbývajících hodnotami kostek. Tento proces pokračuje, dokud nelze provést žádné další tahy nebo dokud nejsou použity všechny kostky.



Obrázek 4.3: Permutace

Mnoho stavů je stejných a vyplývá z různých permutací tahů. Hráč může provádět tahy v jiném pořadí a dosáhnout stejného stavu. To je znázorněno na obrázku 4.3. Každý uzel grafu představuje samostatný stav hry; konečné stavy jsou označeny. Počáteční stav hry (kořenový uzel) má dvě nepoužitá čísla na kostkách. Jeho podřízené uzly představují stavy po použití jednoho čísla na kostce. Nemá smysl je okamžitě vyhodnocovat, protože vedou ke společnému konečnému stavu, ve kterém jsou použita všechna čísla na kostkách. Pro jakoukoli hodnotu kostky, která není dvojitá (jako na obrázku, kde mají kostky různé hodnoty), je počet stavů vedoucích k jednomu společnému stavu $2! = 2$, u dvojitých hodnot je tato hodnota vyšší a v nejhorším případě dosahuje $4! = 24$. Úkolem metody je extrahovat pouze konečné stavy (tj. stavy, kde jsou použity všechny hodnoty kostek nebo neexistují žádné povolené tahy). Všechny takové herní stavy jsou uloženy v samostatném slovníku, kde každý odpovídá posloupnosti tahů, která k němu vede. Stav je složitá struktura, která zabírá relativně velké množství paměťového prostoru. Zároveň je třeba uložit stovky tisíc takových stavů. V tomto případě je použití hašovací tabulky mnohem efektivnější. Takové tabulky se nazývají transpoziční tabulky a v tomto algoritmu se široce používají.

Algoritmus používá hashovací mechanismus, aby zajistil, že jsou prozkoumány pouze jedinečné stavy. Každý stav hrací desky je reprezentován jako řetězový klíč generovaný metodou `HashBoardState`. Tento klíč obsahuje:

- Index každého bodu na desce, který má alespoň jeden kámen.
- Počet kamenů v tomto bodě.
- Barvu majitele.
- Počet kamenů na lajně každého hráče.

Například existuje bod s indexem 5, na kterém jsou 2 černé kameny. Takový bod se zapisuje jako 5:2:B, kde 5 představuje index, 2 počet a B barvu. Jednotlivé body jsou odděleny znakem „|“. Celý klíč má tedy následující formát:

```
1:2:W|2:1:B|6:4:B|8:2:B|12:5:W|13:5:B|17:3:W|19:5:W|24:2:B|0:WBar|0:BBar
```

Samozřejmě je možné tento klíč zkrátit, ale současná struktura umožňuje člověku velmi snadno číst a reprodukovat pozice a nezabírá příliš mnoho paměti. Jak rekurzivní proces prozkoumává nové stavy, každý hashovaný stav se přidává do slovníku nazvaného `stateToMoves`. Tento slovník mapuje hashovaný klíč na odpovídající sekvenci tahů ve formátu `{int src, int dst}`, která daný stav vyprodukovala. Tím se zabrání nadbytečným výpočtům a zaručí se, že různé sekvence vedoucí ke stejnému výsledku nebudou znovu vyhodnocovány. .NET používá kódování UTF-16 [10], takže každý znak v klíči zabírá 2 bajty. Každý nový pár klíč-hodnota ve slovníku tedy zabírá přibližně 220–240 bajtů, takže celý slovník pro třívrstvé prohledávání zřídka zabírá více než 350–400 MB paměti.

Pokud se stav již vyskytl (tj. jeho hash existuje ve slovníku), je přeskočen. Tato technika významně snižuje počet stavů, které musí AI zohlednit, a optimalizuje rozhodovací proces, aniž by byla ohrožena správnost.

Na konci metody má AI seznam jedinečných výsledných stavů hrací desky, z nichž každý je spojen s platnou sekvencí tahů. Tyto stavy slouží jako kandidáti pro další vyhodnocení algoritmem `ExpectiMiniMax`.

ExpectiMiniMax a MinimaxForDiceOutcome

Metoda `ExpectiMiniMax` je jádrem rozhodovacího procesu AI. Jedná se o rekurzivní funkci, která vyhodnocuje očekávanou hodnotu herního stavu a simuluje všechny možné výsledky, které mohou nastat v důsledku hodů kostkami a reakcí hráčů. Tato metoda rozšiřuje tradiční algoritmus `MiniMax` a zpracovává stochastické prvky backgammonu. Tyto uzly náhody představují všechny možné výsledky hodů kostkami, které by mohly nastat v tahu hráče.

Na rozdíl od standardního `MiniMaxu`, který vyhodnocuje jeden deterministický tah na hráče a tah, `ExpectiMiniMax` vyhodnocuje každou možnou kombinaci hodů kostkami. V kontextu backgammonu existuje 21 jedinečných výsledků hodu dvěma šestistrannými kostkami, včetně 15 ne-dvojic (každá se vyskytuje dvakrát kvůli symetrii) a šesti dvojic (každá se vyskytuje jednou). V důsledku toho každá úroveň stromu `ExpectiMiniMax` provádí až 21 nezávislých vyhodnocení potenciálních výsledků.

Tato struktura efektivně zobecňuje algoritmus `MiniMax` tím, že jej volá 21krát za tah. Při každém volání se funkce `MinimaxForDiceOutcome` používá k provedení standardního vyhodnocení `MiniMax` pro konkrétní kombinaci kostek.

Algoritmus 3 Algoritmus ExpectiMiniMax

```
1: funkce EXPECTIMINIMAX(board , depth,maximizingP layer ,currentPlayer)
2:   diceOutcomes  $\leftarrow$  GENERATEALLDICEOUTCOMES
3:   totalValue  $\leftarrow$  0
4:   totalFrequency  $\leftarrow$  0
5:   pro každý dice vdiceOutcomes proved'
6:     frequency  $\leftarrow$  OUTCOMEFREQUENCY(dice )
7:     value  $\leftarrow$  MINIMAXFORDICEOUTCOME(board ,   depth,  maximizingP layer ,
      currentPlayer,dice )
8:     totalValue  $\leftarrow$  totalValue + (frequency    $\times$  value )
9:     totalFrequency  $\leftarrow$  totalFrequency + frequency
10:  konec pro
11:  pokud totalFrequency =0, pak
12:    vrátit pokud maximizingP layer , pak  $-\infty$  jinak  $+\infty$ 
13:  konec pokud
14:  vrátit totalValue/totalFrequency
15: konec funkce
```

Algoritmus 3 představuje skutečný algoritmus použitý v programu jako pseudokód. V tomto algoritmu:

- *diceOutcomes* je seznam všech možných hodů kostkami v backgammonu. Obsahuje 21 jedinečných výsledků, včetně dvojic (např. 4–4) i jiných kombinací (např. 2–5), které jsou generovány pomocnou funkcí *GenerateAllDiceOutcomes*.
- *totalValue* je proměnná používaná k akumulaci váženého součtu všech hodnot vyhodnocení napříč různými výsledky hodu kostkou.
- *totalFrequency* sleduje celkovou váhu, neboli počet, všech vyhodnocených výsledků. Například ne-dvojice se vyskytují s frekvencí 2 kvůli symetrii, zatímco dvojice se vyskytují s frekvencí 1.

MiniMax pro výsledek hodu kostkou

Metoda *MinimaxForDiceOutcome* je pomocná funkce používaná v rámci algoritmu *ExpectiMiniMax*. Zatímco *ExpectiMiniMax* je zodpovědný za zpracování nejistoty prostřednictvím náhodných uzlů (hodů kostkou), tato metoda provádí skutečné deterministické vyhodnocení pro jeden výsledek hodu kostkou. Funguje stejným způsobem jako klasický algoritmus *MiniMax*, střídajíc hráče minimalizující a maximalizující.

Metoda začíná generováním všech jedinečných stavů hrací desky, kterých může aktuální hráč dosáhnout pomocí zadaného výsledku hodu kostkou. Ty se získávají pomocí metody *GenerateUniqueStates*, která simuluje všechny povolené sekvence tahů a pomocí hashování zabraňuje duplicitním stavům.

Algoritmus 4 Minimax pro výsledek hodu kostkou

```
1: funkce MINIMAXFORDICEOUTCOME(board , depth, maximizingP layer,  
   currentPlayer,dice )  
2:   states  $\leftarrow$  GENERATEUNIQUESTATES(board ,currentPlayer ,dice )  
3:   pokud jestates prázdné, pak  
4:     penalty  $\leftarrow$  EVALUATEBOARD(currentPlayer ,board )  
5:     penalty  $\leftarrow$  ifmaximizingP layer thenpenalty  $- 10$  elsepenalty  $+ 10$   
6:     návrat penalty  
7:   konec  
8:   opponent  $\leftarrow$  GETOPPONENT(currentPlayer )  
9:   ifmaximizingP layer then  
10:    value  $\leftarrow -\infty$   
11:    pro každéstate vstates proved'  
12:      childValue  $\leftarrow$  EXPECTIMINIMAX(state depth  $- 1$  false, opponent)  
13:      value  $\leftarrow$  max(value, childValue)  
14:      pokudvalue  $\geq \beta$  , pak  
15:        vrátitvalue  $\triangleleft \beta$  cutoff  
16:      end if  
17:       $\alpha \leftarrow$  max( $\alpha$ , value)  
18:    konec pro  
19:    návrat value  
20:  jinak  
21:    value  $\leftarrow +\infty$   
22:    pro každéstate vstates proved'  
23:      childValue  $\leftarrow$  EXPECTIMINIMAX(state , depth  $- 1$ , true,opponent )  
24:      value  $\leftarrow$  min(value, childValue)  
25:      pokudvalue  $\leq \alpha$  , pak  
26:        návratvalue  $\triangleleft \alpha$  cutoff  
27:      end if  
28:       $\beta \leftarrow$  min( $\beta$ , value)  
29:    konec pro  
30:    návrat value  
31:  konec if  
32: konec funkce
```

Součásti algoritmu fungují následovně:

- Volání funkce `GenerateUniqueStates` vygeneruje všechny platné herní stavy, které lze dosáhnout z aktuální pozice na hrací desce. Pokud je seznam stavů prázdný, znamená to, že hráč nemá k dispozici žádné povolené tahy. V tomto případě algoritmus uplatní penalizaci, aby simuloval negativní dopad vynechání tahu.
- Proměnná `opponent` uchovává protivníka, na jehož tah bude v rekurzi následovat.

Algoritmus vrací nejlepší skóre nalezené mezi všemi podřízenými stavy, v závislosti na tom, zda se jedná o úroveň maximalizace nebo minimalizace.

Důležité je, že pokud nejsou nalezeny žádné povolené tahy, je to považováno za vynechaný tah. V tomto případě se vypočítá aktuální hodnocení hrací desky, což bude podrobněji popsáno

podrobněji v oddíle 6. Namísto vrácení neutrální hodnoty algoritmus aplikuje malou penalizaci, aby odrážel nevýhodu ztráty tahu. V průměru hráč backgammonu postoupí přibližně o 8,166 bodů na tah (podrobněji v příloze B). Navíc vynechání tahu může vystavit kameny zásahům soupeře. Proto se aplikuje zaokrouhlená penalizace 10 bodů, aby se simuloval dopad vynechání tahu.

Jsou-li k dispozici povolené tahy, metoda rekurzivně volá `ExpectiMiniMax` na každém výsledném stavu

výsledný stav hrací desky a přepne roli hráče (z maximalizace na minimalizaci nebo naopak).

Metoda používá prořezávání Alpha-Beta, což je zohledněno v pseudokódu.

Po vyhodnocení všech možných výsledných stavů pomocí algoritmu `ExpectiMiniMax` vybere AI ten s nejvyšší očekávanou hodnotou. Každý stav hrací desky je spojen s jedinečnou sekvencí tahů, která byla dříve uložena během generování tahů pomocí slovníku `stateToMoves`

. Tento slovník mapuje hashované reprezentace stavů hrací desky na přesnou sekvenci tahů, které k nim vedly. Jakmile je identifikován nejlepší stav hrací desky, AI načte odpovídající sekvenci tahů pomocí metody `RetrieveMoveSequence`.

Samotné vyhodnocení se provádí pomocí samostatné vyhodnocovací funkce, která vrací celočíselnou hodnotu odhadující, jak příznivý je daný stav hry pro aktuálního hráče. Podrobnosti a struktura této vyhodnocovací funkce budou popsány později v kapitole 6.

Nakonec je sekvence tahů prováděna krok za krokem pomocí stejných veřejných metod, jaké by byly volány pro lidského hráče. To zajišťuje plnou shodu s pravidly hry a umožňuje AI hrát transparentně v rámci stávající herní smyčky. Každý tah je zpracován pomocí stejných rutin pro ověření a aktualizaci hracího pole, takže je zachována konzistence mezi interakcemi AI a uživatele.

Kapitola 5

Optimalizace algoritmu ExpectiMiniMax

Algoritmus ExpectiMiniMax je výpočetně náročný kvůli své rekurzivní povaze a mnoha možným herním stavům, které je třeba vyhodnotit. Jelikož se každý tah rozšiřuje do více větví v závislosti na všech možných výsledcích hodu kostkou, počet stavů roste exponenciálně s hloubkou prohledávání. V praxi to způsobuje, že je algoritmus pomalý a náročný na paměť. Například algoritmus *-MiniMax je mnohem více optimalizovaný a dokáže prohledávat do hloubky 5 tahů, ale vyžaduje normalizaci hodnotící funkce [6].

Celková doba výpočtu závisí na dvou klíčových faktorech: počtu herních stavů, které je třeba vyhodnotit, a době potřebné k vyhodnocení jednoho stavu. Bylo zavedeno několik optimalizačních strategií, aby byla AI během hry responzivní a použitelná. Tyto strategie se zaměřují na snížení počtu stavů, které je třeba zpracovat, zrychlení vyhodnocování stavů a lepší využití hardwarových schopností, jako je multithreading.

Tato kapitola pojednává o hlavních optimalizacích použitých při implementaci. Každá podkapitola představuje jiný přístup ke zlepšení výkonu.

5.1 Časová složitost a potřeba optimalizace

Každé rozhodnutí učiněné AI zahrnuje simulaci mnoha možných budoucích stavů, jejichž počet se s hloubkou prohledávání rychle zvyšuje.

V backgammonu je jeden tah hráče ovlivněn výsledkem hodu dvou kostek. Při zohlednění symetrických hodnot a dvojic existuje 21 různých výsledků hodu. Počet povolených herních stavů, kterých může hráč dosáhnout pro každý výsledek, se liší, ale v praxi se v průměru pohybuje kolem 19–20 stavů na jeden výsledek. Pro jeden tah (tj. kompletní tah jednoho hráče) lze tedy počet možných výsledných herních stavů odhadnout takto:

$$N_1 = 21 \times 20 = 420$$

Při hodnocení hloubky 3 tahů – jeden tah AI, jeden tah soupeře a jedna další odpověď AI – se počet herních stavů, které je třeba prozkoumat, na každé úrovni násobí. Za předpokladu podobných faktorů větvení v každém tahu lze celkový počet stavů odhadnout jako:

$$N_{(\text{celkem})} = (21 \times 20)^3 = 420^3 = 74\,088\,000$$

To znamená, že k úplnému vyhodnocení herního stromu v hloubce 3 tahů může algoritmus potřebovat zpracovat přes 74 milionů různých stavů na šachovnici, z nichž každý vyžaduje generování tahu, simulaci

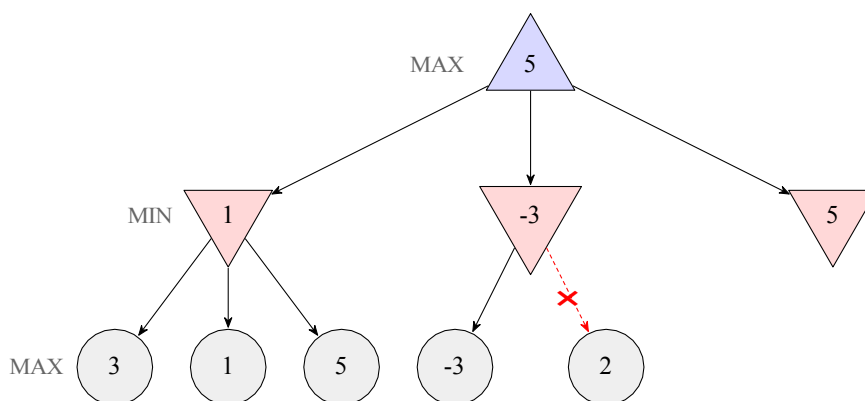
ulaci a vyhodnocení. Tyto výpočetní náklady činí rozhodování v reálném čase téměř nemožným bez dalších optimalizací. V praxi počet stavů dosahuje asi 5 milionů a doba výpočtu dosahuje 9 minut.

Kromě toho musí být pro každý koncový uzel (list) ve stromu vypočítáno numerické hodnocení pomocí heuristické hodnotící funkce. Jelikož je tato funkce volána jednou pro každý listový uzel a potenciálně stovky tisíc krát během kompletního vyhodnocení, má její výkonnost významný dopad na celkovou dobu běhu AI.

Vzhledem k této exponenciální časové složitosti je nezbytné implementovat různé optimalizační techniky, aby se snížil počet zvažovaných stavů a zrychlily jednotlivé výpočty. Tyto techniky – jako je prořezávání alfa-beta, ukládání stavů do mezipaměti a multithreading – jsou popsány v následujících částech.

5.2 Prořezávání vyhledávacího stromu s tříděním stavů

V algoritmu MiniMax není nutné prozkoumat všechny větve; vynechání některých z nich může zkrátit dobu hledání, aniž by se změnil výsledek. Technika odstraňování zbytečných větví se nazývá **prořezávání alfa-beta**. [11, oddíl 5.3]



Obrázek 5.1: Příklad prořezávání alfa-beta

Strom na obrázku 5.1 ilustruje, jak funguje prořezávání alfa-beta na tříúrovňovém herním stromu se střídajícími se hráči MAX, MIN a MAX.

V kořenu, který je uzlem typu MAX, algoritmus začíná s počátečními hodnotami $\alpha = -\infty$ a $\beta = +\infty$. Nejprve prozkoumá nejlevější podstrom MIN. Tento uzel MIN má tři potomky, z nichž každý představuje možný tah MAX, který vrací hodnotu 3, 1 a 5. MAX je použije k aktualizaci své interní hodnoty α . Po zpracování všech potomků vybere MIN mezi nimi minimum, kterým je 1, a vrátí ho ke kořenu. Kořenový uzel MAX pak aktualizuje svou aktuální nejlepší hodnotu na $\alpha = 1$.

Dále se algoritmus přesune k druhému podstromu MIN. Hodnota jeho prvního potomka (uzlu MAX) je 3, takže aktuální nejlepší hodnota MIN se změní na 3 a hodnota β se odpovídajícím způsobem aktualizuje. Než však dojde k vyhodnocení druhého potomka tohoto uzlu MIN, algoritmus zkontroluje, zda je nutné pokračovat v prohledávání. Jelikož aktuální hodnota MIN $\beta = 3$ je již menší nebo rovna hodnotě $\alpha = 1$ (dosavadní nejlepší hodnota MAX), je druhý potomek okamžitě vyřazen. Není třeba jej vyhodnocovat, protože MAX by nikdy nezvolil cestu, která by soupeři umožnila vynutit horší výsledek, než jaký již byl zaznamenán (nejlevější potomek s hodnotou 1).

Po prořezání algoritmus pokračuje k finálnímu podstromu MIN na pravé straně, který obsahuje jediný uzel MAX vracující hodnotu 5. Tato hodnota je předána zpět ke kořenu a MAX ji porovná s předchozí nejlepší hodnotou. Jelikož 5 je větší než 1, MAX aktualizuje svůj konečný výsledek na 5.

Jak ukazuje tento příklad, prořezávání Alpha-Beta umožnilo algoritmu přeskočit vyhodnocení část stromu, aniž by to ovlivnilo výsledek. Klíčové rozhodnutí o prořezání bylo založeno na nerovnosti α , kde aktuální hodnota MIN nemohla vést k lepšímu výsledku pro hráče MAX. To činí prořezávání Alpha-Beta nezbytnou optimalizací pro efektivní adversariální vyhledávání.

5.2.1 Třídění tahů

Prořezávání Alpha-Beta není konzistentní technikou. V nejlepším případě bude časová složitost algoritmu $O(b^m)$. Zároveň se v nejhorším případě časová složitost nebude lišit od klasického MiniMaxu. Prořezávání Alpha-Beta je velmi citlivé na pořadí tahů. Z pohledu uzlu MAX, pokud má nejlevější potomek vysokou hodnotu (pravděpodobně dobrý tah), pak bude většina následujících větví odříznuta a algoritmus se přiblíží časové složitosti nejlepšího případu.

Proto je před vyhodnocením generovaných herních stavů nutné je seřadit tak, aby byly nejslibnější tahy vyhodnoceny jako první. Takové tahy by měly přinášet strategickou výhodu alespoň na první pohled. Princip, podle kterého jsou tahy seřazovány, by neměl být příliš složitý. Nemá smysl vyhodnocovat tah velmi podrobně, pokud je již dobře vyhodnocen v maximální hloubce, protože to pouze zpomalí algoritmus a zmaří smysl třídění. Relativně jednoduchým způsobem, který byl implementován, je vyhodnotit nejprve všechny **tahy, při nichž aktuální hráč porazil soupeřovu figurku**. To dobře využívá následnou vyhodnocovací funkci, která zohledňuje vzdálenost figurek od okraje hrací plochy. Tyto tahy nejsou vždy dobré, protože někdy otevírají prostor pro protihru soupeře, ale stačí k vyřazení většiny tahů, které v budoucnu nepřinesou žádnou strategickou výhodu.

Obecně lze říci, že pouze použití prořezávání Alpha-Beta umožnilo snížit počet stavů, které je třeba vyhodnotit, o 10–12krát ve srovnání s neoptimalizovanou verzí v současné implementaci hry. Doba hledání se snížila na 1 minutu na tah pro hru o 3 tazích, což je stále nepřiměřeně dlouhá doba a vyžaduje další optimalizace.

5.3 Optimalizace hodnotící funkce

Hodnotící funkce hraje v algoritmu ExpectiMiniMax klíčovou roli, protože určuje heuristickou hodnotu herního stavu, když vyhledávání dosáhne listového uzlu. Vzhledem k větvicímu faktoru a mnoha možným výsledkům hodu kostkami v backgammonu může být hodnotící funkce během jediného rozhodovacího cyklu spuštěna stovky tisíc nebo dokonce milionykrát. Optimalizace této funkce je proto nezbytná pro udržení přiměřeného výkonu. Úkolem optimalizace je co nejvíce zkrátit kód a vyhnout se různým faktorům, které ovlivňují rychlost jeho provádění.

Je možné najít několik klíčových faktorů pro optimalizaci evaluační funkce v C#:

- **Vyhýbání se konstrukcím vyšší úrovně.** Například `foreach` je čitelný a má velmi čistou syntaxi, ale zavádí režii iterátoru, zejména při použití na pole. Je mnohem lepší nahradit tuto konstrukci tradiční smyčkou `for` – například `for (int i = 0; i < 24; i++)`. Eliminuje to vytvoření objektu enumerátoru a zlepšuje výkon.
- **Omezení opakovaného přístupu k vlastnostem.** Přiřazení často používaných vlastností k lokálnímu proměnným, jako je například ukládání `board.Points` do reference `var points`, snižuje režii volání funkcí a zabraňuje nadbytečnému přístupu k vlastnostem v rámci smyček.
- **Vyhýbání se zbytečným alokacím.** Funkce jako LINQ zavádějí skryté alokace a zpožděné provádění. V kódu, kde je výkon kritický, je lepší zabránit nadbytečnému využití paměti.

Tento problém se týká programovacích jazyků vyšší úrovně a při psaní takového kritického kódu je nutné být co nejefektivnější.

Příklad: Vyhodnocení vzdálenosti desky

```
for (int i = 0; i < 24; i++)
{
    var point = board.Points[i];
    if (point.Owner == CheckerColor.White)
        whiteDistance += (25 - i) * point.Checkers.Count;
    if (point.Owner == CheckerColor.Black)
        blackDistance += i * point.Checkers.Count;
}
```

Tento úryvek kódu je součástí implementované vyhodnocovací funkce, která vypočítává celkové vzdálenosti kamenů pro každého hráče.

Podmínkové příkazy uvnitř těsných smyček – zejména vnořené bloky `if-else` – zavádějí větvení, které může narušit schopnost procesoru předvídat následující instrukce. Moderní procesory se při zamezování zastavení pipeline značně spoléhají na předvídaní větvení. Pokud je podmínka větvení nepředvídatelná (např. se mezi iteracemi často mění), procesor se splete v předpovědi a vyprázdní pipeline, což vede ke snížení výkonu. Je důležité používat takové podmíněné příkazy co nejméně. Navíc, pokud je nějaká hodnota známou konstantou, například počet bodů na hrací ploše, je mnohem lepší použít konstantní číslo namísto volání stejné funkce `point.Count()`.

Vzhledem k tomu, že se vyhodnocovací funkce volá tak často, může optimalizace i jejích malých částí přinést významné zvýšení výkonu. Tyto mikrooptimalizace mají kumulativní účinek a společně umožňují AI vypočítat optimální tah mnohem rychleji. Ušetřený čas závisí na složitosti samotné vyhodnocovací funkce, proto by tato funkce měla být co nejjednodušší, aby bylo provedení co nejrychlejší.

5.4 Paralelní zpracování výsledků hodu kostkou

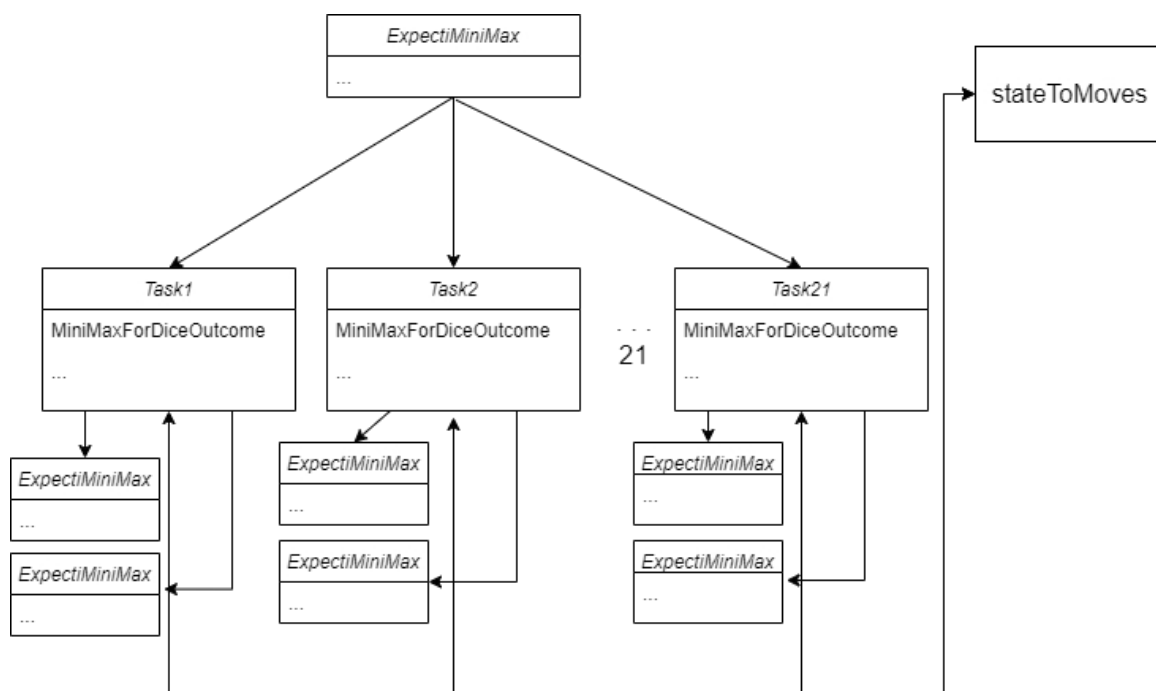
Následující optimalizace byla inspirována a částečně vychází z [8].

V algoritmu ExpectiMiniMax jsou všechny výsledky hodu kostkou modelovány jako uzly náhody a jejich podřízené uzly představují všechny možné reakce na konkrétní hod kostkou.

Jelikož výsledek jednoho hodu kostkou neovlivňuje ostatní, jsou tyto větve vyhledávacího stromu vzájemně nezávislé. V důsledku toho lze vyhodnocení každého výsledku hodu kostkou provádět paralelně. Tato myšlenka je v metodě ExpectiMiniMax implementována spuštěním samostatného úkolu pro každý z 21 výsledků hodu kostkou. Tyto úkoly souběžně vypočítávají očekávanou hodnotu pro každý uzel náhody.

- Každý úkol spouští metodu `MinimaxForDiceOutcome`, která provádí úplné vyhodnocení min-max pro jeden hod kostkou.
- Všechny úlohy jsou shromážděny do seznamu a prováděny paralelně pomocí `Task.Run()`.
- Po dokončení všech úloh se vážený průměr jejich výsledků použije k výpočtu konečné očekávané hodnoty.

Tato paralelita umožňuje AI využívat více procesorových jader a výrazně zkracuje čas potřebný ke zpracování hlubokých stromů, zejména když počet stavů roste exponenciálně s hloubkou prohledávání.



Obrázek 5.2: Struktura úkolů

Obrázek 5.2 ukazuje schéma vztahové struktury mezi metodami ExpectiMini-Max a MiniMax, kde každý MiniMax je ve svém vlastním úkolu. Tato metoda používá slovník `stateToMoves` ke čtení a zápisu nových hashovaných stavů hrací desky. Metody MiniMax volají ExpectiMiniMax, který zase může vytvářet nové úkoly.

Tato struktura vede k určitým problémům, proto musí být implementována velmi pečlivě. Největšími problémy jsou:

- **Sdílená paměť**

Ačkoli tato technika výrazně zrychluje výpočty, měla by být implementována s opatrností, protože takové asynchronní procesy jsou při práci se sdílenou pamětí poměrně nebezpečné. ExpectiMiniMax používá tři taková paměťová místa: Alpha, Beta a slovník stateToMoves. Alpha a Beta se mění relativně zřídka, na rozdíl od slovníku, kde každý úkol neustále čte a zapisuje nové hashované stavy. Aby se předešlo konfliktům, je implementován jako ConcurrentDictionary, což zajišťuje přístup bezpečný pro vlákna.

- **Počet úloh**

Vytváření nových úloh uvnitř rekurzivních metod s větší opatrností je nutné, protože jejich počet roste exponenciálně. Nejlepším řešením je používat je v jedné nebo maximálně dvou nejhlubších úrovních stromu. Zapojení více úrovní vytvoří absurdně velký počet úloh a jejich umístění blízko kořene stromu uměle vytvoří strom pro prohledávání do šířky a eliminuje podstatu Alpha-Beta a MiniMax, které vyžadují strom pro prohledávání do hloubky.

Při tomto způsobu využití úloh se sdílená paměť stává úzkým hrdlem, protože úlohy stále musí čekat na svůj pořad, aby mohly zapsat nová data, a nemohou běžet zcela odděleně. Tento efekt je však relativně mírný a má jen malý vliv na rychlost výpočtu. Omezením možnosti vytvářet nové úlohy na posledních dvou úrovních stromu bude průměrný počet úloh asi 100; nikdy nedosáhne 2^{12} . Generování nových stavů hrací desky a jejich hashování zabere relativně více času než zápis do slovníku. Klíče jsou zaručeně jedinečné téměř ve všech případech. Jedinou výjimkou je pozdní fáze hry, kdy některý z hráčů vyrazuje kameny. V této situaci je faktor větvení minimální, takže ani pokus o zápis stejného klíče různými úlohami nemá nepříznivý vliv na rychlost.

5.5 Selektivní prohlubování

Třídění tahů podle míry přínosu a strategického dopadu pomáhá technice prořezávání Alpha-Beta odříznout výrazně větší části stromu a činit tak častěji, ale zároveň tento přístup znamená procházení všech větví stromu. Pro další optimalizaci a snížení počtu uzlů, které je třeba prozkoumat, lze použít metodu selektivního prohlubování. Velmi podobnou myšlenku popsali Buro a aplikovali ji na hru Othello. Jeho metoda ProbCut [4] navrhuje statistické prořezávání založené na předpovědích mělkého vyhodnocení.

Základní myšlenkou navrhované optimalizace je využít pozorování, že v praxi silní lidští hráči nehodnotí každý možný tah stejně. Místo toho se instinktivně soustředí na tahy, které se na základě zkušeností a intuice ohledně hrací desky jeví jako slibnější. Intuice je poměrně vágní pojem, a proto musí být nahrazena heuristickým algoritmem pro třídění možných tahů. Při převedení do výpočetního modelu algoritmus třídí dostupné tahy pomocí heuristické hodnotící funkce a předpokládá, že lepší tahy se s větší pravděpodobností objeví dříve v seznamu. Hodnocením pouze podskupiny těchto nejlépe hodnocených tahů – obvykle levé poloviny seznamu tahů – lze ořezat významné části vyhledávacího stromu, což vede ke snížení počtu uzlů.

Logiku tohoto přístupu lze popsat v následujících krocích:

1. Pomocí heuristické funkce seřadíme dostupné tahy tak, aby ty slibnější byly v seznamu na prvních místech.
2. Simulujte velké množství her a analyzujte, v které části seznamu tahů se obvykle objevuje nejlepší tah. Vyberte mezní percentil, který zachytí nejlepší tahy ve většině případů.
3. Během skutečného vyhledávání prozkoumejte pouze tahy v rámci vybraného horního percentilu a zbytek ignorujte. Tím se sníží počet uzlů, které musí algoritmus vyhodnotit.

Tento přístup funguje podobně jako beam search, kde se rozšiřuje pouze omezený počet nejlepších kandidátů. Nezaručuje, že bude vždy zohledněn nejlepší tah, ale umožňuje algoritmu prohledávat slibné oblasti hlouběji. Může výrazně zrychlit běh algoritmu tím, že sníží počet tahů, které během hledání zkoumá. Protože algoritmus neztrácí čas na zjevně slabé tahy, může se dostat hlouběji do herního stromu. To mu může pomoci najít lepší dlouhodobé strategie. Hranice pro prořezávání může být založena na reálných datech ze simulovaných her, což činí rozhodnutí informovanějším, namísto toho, aby bylo založeno na dohaděch.

Tento přístup má však i své slabiny:

- Tato metoda neprochází všechny možné tahy, a proto může opomenout důležité tahy, které heuristika nehodnotí jako příliš výhodné. Pokud není heuristika spolehlivá, může to vést k chybám. Pokud třídění tahů funguje dobře v blízkosti kořene stromu, ale s hloubkou stromu se zhoršuje, může se prohledávání s postupujícím hloubkou oslabovat.
- Nejlepší mezní percentil se může měnit v závislosti na fázi hry nebo situaci na šachovnici, takže jeho jednorázové nastavení nemusí fungovat dobře pro všechny pozice.

Ačkoli tato metoda striktně nesleduje standardní logiku minimaxu, může být užitečným kompromisem mezi přesností a rychlostí. V situacích, kdy jsou rychlá rozhodnutí důležitější než dokonalá přesnost, může tato technika nabídnout praktickou výhodu.

Toto téma je kontroverzní a vyžaduje mnohem více výzkumu, proto nebyl tento přístup implementován. Technika je založena na skutečnosti, že existuje určitý heuristický algoritmus, který přesně eliminuje všechny „slabé“ tahy. Za předpokladu, že takový algoritmus již existuje, jeho použití na prořezávání Alpha-Beta přiblíží časovou složitost co nejblíže k nejlepšímu možnému případu. Pokud ano, o kolik bude vyhledávání rychlejší, když se ignorují některé větve ve srovnání s prořezáváním Alpha-Beta? Pokud takový heuristický algoritmus neexistuje, může ignorování větví vést k promarnění šance na výhru. Obecně lze tento přístup implementovat po hlubším výzkumu a pravděpodobně umožní, aby ExpectiMiniMax pracoval v hloubce 4 tahů.

Kapitola 6

Hodnotící funkce, faktory ovlivňující hru

6.1 Hodnotící funkce

Hodnotící funkce je nejdůležitější součástí algoritmu ExpectiMiniMax. Nezáleží na tom, jak hluboko vyhledávání sahá, ani jak rychle jsou stavy vyhodnocovány, pokud hodnotící funkce nehodnotí pozice dobře a neodráží skutečnou situaci na šachovnici. Algoritmus ExpectiMiniMax má známé limity, takže prakticky jediným způsobem, jak zlepšit jeho kvalitu, je vytvořit co nejlepší hodnotící funkci.

V AI pro soupeřivé hry hodnotící funkce odhaduje, jak příznivý je daný stav na šachovnici pro hráče, když vyhledávání dosáhne nekonečného uzlu. V algoritmu ExpectiMiniMax se tato funkce volá na listových uzlech vyhledávacího stromu, což znamená, že může být volána stovky tisícrát během výpočtu jediného tahu. Z tohoto důvodu musí být jak výpočetně jednoduchá, tak strategicky účinná. Pomalá funkce se může stát významným úzkým hrdlem, zatímco nepřesná funkce může vést k tomu, že AI bude pozice soustavně špatně hodnotit.

Hodnocení je založeno na počtu bodů. Počet bodů měří vzdálenost, kterou musí kameny hráče urazit, aby dosáhly domácího pole a mohly být vyřazeny. Nižší počet pipů obecně odpovídá silnější pozici. Myšlenka použití počtu pipů jako základu pro hodnocení pozic byla inspirována článkem na Backgammon Galaxy [1], kde se počet pipů používá jako indikátor toho, kdo ve hře vede. Tato práce rozšiřuje tento koncept formalizací hodnocení jako rozdílu mezi celkovým počtem pipů hráče a soupeře.

Spoléhat se pouze na jeden parametr není řešením, proto hodnotící funkci rozšiřují modulární faktory. Tyto faktory upravují základní hodnocení bodů zohledněním dalších pozičních a taktických prvků. Každý faktor lze ručně aktivovat a přispívá k multiplikativní úpravě skóre hráče založeného na bodech. Tento design umožňuje snadné testování různých strategických přístupů a vývoj herního stylu AI bez přepisování základní logiky.

Modularita tohoto systému podporuje experimentování a ladění výkonu. Oddělení konkrétních faktorů od základního hodnocení usnadňuje izolaci účinků různých strategických komponent a jejich nezávislou optimalizaci.

Počet bodů se pro každého hráče vypočítá sečtením bodů, o které se musí každá kamenka posunout ze své aktuální pozice do bodu vyřazení. U kamenek na barce je vzdálenost k bodu vyřazení konstantní a rovná se 25. Hodnotící funkce pak

vypočítá rozdíl mezi počtem bodů soupeře a počtem bodů aktuálního hráče. Je možné tuto funkci formálně definovat.

Hodnotící funkce backgammonu

Nechť B označuje aktuální stav hrací desky a nechť $P \in \{\text{White}, \text{Black}\}$ označuje aktuálního hráče. Definujme:

- c_i^W : Počet bílých kamenů na bodu i
- c_i^B : Počet černých kamenů na bodu i
- b^W : Počet bílých kamenů na bar
- b^B : Počet černých kamenů na bar

Hrací deska se skládá z 24 bodů označených indexem $i = 1$ až 24, kde:

- Bod $i = 1$ je nejbližší domácí metě pro černé
- Bod $i = 24$ je nejbližší domácí základně bílých

Počet bodů pro každého hráče:

$$D_W = 25 \cdot b^W + \sum_{i=1}^{24} (25-i) \cdot c_i^W$$

$$D_B = 25 \cdot b^B + \sum_{i=1}^{24} i \cdot c_i^B$$

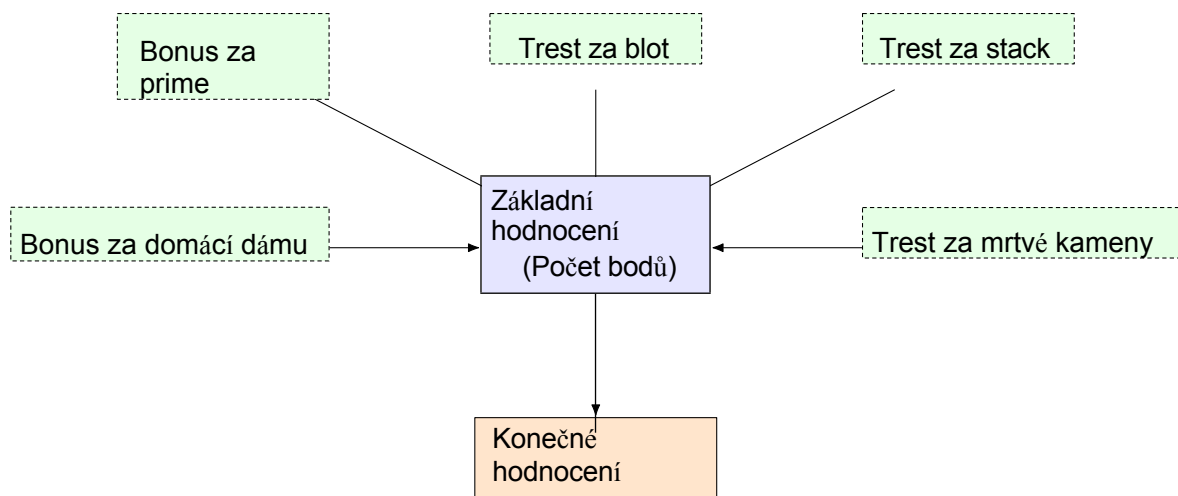
Zde D_W a D_B představují celkový počet bodů (vzdálenost k vyřazení) pro bílé a černé. Funkce $E(P, B)$ vrací kladnou hodnotu, když je hráč P ve výhodné pozici (tj. blíže k vyřazení), a zápornou hodnotu, když je pozadu.

Hodnotící funkce:

$$E(P, B) = \begin{cases} D_B - D_W & , \text{pokud } P = \text{bílý} \\ D_W - D_B & , \text{pokud } P = \text{černá} \end{cases}$$

Funkce se nezakládá pouze na počtu bodů aktuálního hráče, ale také na počtu bodů jeho soupeře. Jelikož backgammon je hra s nulovým součtem, jakékoli zvětšení vzdálenosti soupeře od okraje hrací desky nepřímo přibližuje aktuálního hráče k vítězství. Vzhledem k tomu, že podle tohoto hodnocení je každý tah stejný, protože kamen se vždy pohybuje o počet bodů daný kostkami, vzdálenost od okraje hrací desky se zmenšuje o stejnou hodnotu. Rozdíl ve volbách spočívá v tazích, při nichž kamen jednoho hráče zasáhne kamen druhého hráče. AI volí tahy, při nichž bude mít v budoucnu nejvíce příležitosti k úderu a soupeř nejméně. To vede AI k poměrně agresivní hře, což je naprosto správný přístup, ale zároveň to není zárukou vítězství. Když se prohledávání zastaví v určité hloubce, nebere v úvahu potenciální další tahy soupeře a silné úvodní tahy mohou v budoucnu vést k prohrané pozici. Funguje to i opačně – mírně agresivní tahy, které v určité hloubce vytvářejí obrannou formaci, jsou stabilnější a obvykle přinášejí v budoucnu více výhod.

6.2 Faktory



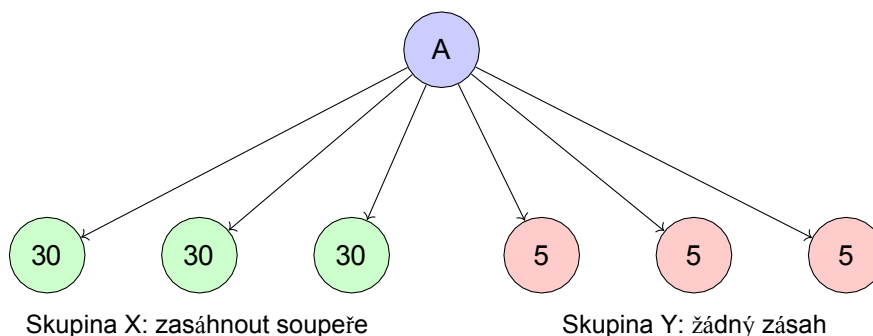
Obrázek 6.1: Základní hodnotící funkce a hlavní faktory

Základní hodnocení je poměrně soběstačná funkce, má však své nevýhody způsobené příliš agresivní hrou a relativně malou hloubkou prohledávání. Sílu pozice ovlivňuje mnoho různých faktorů, takže jejich přidání do základního hodnocení posiluje algoritmus a kompenzuje jeho nevýhody.

Funkce základního hodnocení v systému umělé inteligence pro backgammon se opírá o počet bodů. Ačkoli takové hodnocení účinně odráží celkový průběh hry, má jednu významnou nevýhodu: většina pozic je z hlediska bodů považována **za rovnocennou**, i když se strategicky velmi liší.

Vezměme si například pozici 4, ze které se v hloubce 3 vyskytují dvě skupiny výsledných pozic: skupina X , ve které hráč během sekvence zasáhne soupeřovu kamenku, a skupina Y , ve které nejsou zasázeny žádné kamenky. Pokud je hodnocení založeno výhradně na počtu bodů, pak všechny pozice ve skupině X budou mít stejné skóre a všechny pozice ve skupině Y budou mít stejné skóre. V důsledku toho systém nerozpozná strategickou výhodu akcí, jako je stavění bloků a primů, nebo vyhýbání se blotům a mrtvých kamenů.

Kromě toho se každý tah v backgammonu často skládá z více kroků, které společně mění stav na hrací desce. Některé z těchto kroků mohou pozici posílit, zatímco jiné ji mohou vystavit riziku. Bez dalších faktorů nedokáže hodnotící funkce rozlišit silné tahy od slabých. To vede k efektu **umělého zploštění** v hodnotícím prostředí, což snižuje kvalitu rozhodnutí.



Obrázek 6.2: Problém hodnocení

Obrázek 6.2 tento problém jasně ilustruje. **Přidání specializovaných faktorů** — jako jsou penalizace za odkryté kameny, bonusy za přímý nebo kameny v domácí desce — umožňuje, aby se hodnotící funkce stala citlivou na strategické detaily. Podporuje tahy, které vedou ke strukturálním výhodám, a penalizuje riskantní akce, i když celkový počet bodů zůstává nezměněn.

Pro určení nejrelevantnějších faktorů byla odehrána série her, ve kterých AI vedla jednoho hráče s základní evaluací proti hráči vedenému AI se základní evaluací, včetně jediného faktoru popsaného v sekci 6.3.

Faktor blot

Faktor blot byl zaveden do hodnotící funkce, aby řešil kritickou strategickou zranitelnost v backgammonu: přítomnost nechráněných jednotlivých kamenů, neboli blotů. K blotu dochází, když hráč ponechá přesně jeden kámen na bodě, který může být snadno zasažen soupeřem. Základní hodnocení takové situace nerozpoznává a hodnotí pozice s mnoha bloty stejným způsobem jako ekvivalentní pozice bez nich, což zhoršuje kvalitu daného skóre pro stav hry. Formálně lze faktor blot vyjádřit následovně: Necht' s je základní hodnotící skóre (např. počet bodů) pro danou pozici hráče. **Faktor blot** zavádí multiplikační penalizaci založenou na počtu **blotů**, které jsou definovány jako body obsazené přesně jedním kamenem daného hráče.

Necht' b označuje počet blotů hráče na desce. Upravené skóre hodnocení s' se vypočítá jako:

$$s' = s \cdot (1 + \alpha \cdot b)$$

kde $\alpha > 0$ je pevný penalizační koeficient za každý blot. V současné implementaci je $\alpha = 0,03$. Každý blot zvyšuje skóre efektivní vzdálenosti o 3 %, čímž se daná pozice jeví jako slabší. To motivuje AI, aby se vyhýbala vytváření exponovaných pozic s jedinou figurkou, které jsou zranitelné vůči úderům.

Pokud má hráč skóre $s = 100$ a $b = 3$ blotů, upravené skóre bude:

$$s' = 100 \cdot (1 + 0,03 \cdot 3) = 100 \cdot 1,09 = 109$$

Tato penalizace se aplikuje na počet bodů aktuálního hráče, nikoli na konečný výsledek. Skóre je tak konzistentní a nezávisí na rozdílu v bodech mezi hráčem a soupeřem. Mnohem efektivnějším řešením je navíc zkontrolovat, zda je blot pro soupeře dosažitelný, a pokud ano, zjistit, jaká je šance na zásah. Tento přístup by však vyžadoval více výpočetních zdrojů než základní vyhodnocení, proto nebyl použit.

Trest za hromadění kamenů

Dalším problémem, který může nastat, je hromadění kamenů na stejném bodě. K zablokování bodu stačí pouze dva kameny, aby to bylo účinnější. Algoritmus proto musí kameny rozložit rovnoměrněji a kontrolovat více pozic. Za tímto účelem byl přidán faktor hromadění kamenů.

Necht' s je základní hodnota hodnocení (např. počet bodů) pro danou pozici hráče. **Trest za nahromadění kamenů** zavádí multiplikační trest založený na počtu kamenů naskládaných nad přijatelnou hranici na jakémkoli jednotlivém bodě.

Necht' je každý bod na desce označen jako P_i a necht' c_i je počet kamenů, které hráč vlastní na bodu P_i . Necht' T je prahová hodnota pro naskládání; v této implementaci je $T = 5$. Definujme celkový počet nadměrných naskládaných kamenů e jako:

$$e = \sum_{i=1}^{24} \max(0, c_i - T)$$

Upravené hodnotící skóre' se vypočítá jako:

$$s' = s \cdot (1 + \alpha \cdot e) \text{ kde } \alpha$$

$\alpha = 0,02$ je penalizační koeficient za každý nadbytečný kámen.

Každý kámen nad prahovou hodnotou přidává 2% penalizaci ke skóre hráče, což odrazuje od neefektivní koncentrované formace a podporuje rozloženou kontrolu po celé desce.

Předpokládejme, že hráč má $s = 120$ a má 3 body s 7, 6 a 5 kameny.

Přebytek je $(7 - 5) + (6 - 5) + (5 - 5) = 2 + 1 = 3$. Pak:

$$s' = 120 \cdot (1 + 0,02 \cdot 3) = 120 \cdot 1,06 = 127,2$$

Je důležité zabránit záporné penalizaci, protože by to vedlo k situaci, kdy by byl podporován vysoký počet blotů, takže penalizace je 0 nebo kladné číslo, které se vybere ve funkci max.

Trest za mrtvé kameny

Mrtvé kameny jsou kameny umístěné na posledních dvou polích jejich domácího pole (nejblíže k zóně vyřazení). Omezují počet taktických možností, které má hráč k dispozici. Mrtvé kameny se nemohou účastnit útočných akcí, jako je srážení nebo vytváření primů, a nepřispívají k ovládnutí středu nebo soupeřovy strany hrací desky. V důsledku toho jsou v podstatě nečinné a snižují dynamickou hodnotu pozice. Trestání nadbytečných kamenů na těchto pozicích odrazuje od příliš pasivního hromadění kamenů v rané fázi hry a podporuje aktivnější rozložení kamenů. Je naprosto správné snažit se blokovat první dva body, problém nastane pouze tehdy, když je na nich příliš mnoho kamenů. V praxi se mnoho her protáhne, protože jeden hráč od samého začátku dominoval na desce a rychle přesunul všechny své kameny až na několik výjimek na konec desky, a jelikož všechny ostatní body ovládá soupeř, má hráč malou šanci dostat posledních pár kamenů do cíle. Soupeř této příležitosti využije a následně vyrovná pozici. Vítězná hra se tak může protáhnout a v nejhorším případě dokonce skončit prohrou.

Necht' s je základní hodnotící skóre (např. počet bodů) pro pozici daného hráče. Necht' D bude množina indexů pro dvě mrtvé zóny:

$$D = \begin{cases} \{1, 2\}, & \text{pokud je hráč černý} \\ \{23, 24\}, & \text{pokud je hráč bílý} \end{cases}$$

Nechť c_i je počet kamenů, které hráč vlastní na bodě i . Nechť $T = 2$ je prahová hodnota tolerance na bod. Pak je celkový přebytek mrtvých kamenů:

$$d = \sum_{i \in D} \max(0, c_i - T)$$

Upravené hodnotící skóre s' se vypočítá jako:

$$s' = s \cdot (1 + \beta \cdot d)$$

kde $\beta = 0,03$ je penalizační koeficient za každou nadbytečnou mrtvou kamenku.

Předpokládejme, že hráč má $s = 110$ a celkem 5 kamenů na bodech 1 a 2, z toho 3 na bodě 1 a 2 na bodě 2. Pak:

$$d = \max(0, 3 - 2) + \max(0, 2 - 2) = 1 + 0 = 1$$

$$s' = 110 \cdot (1 + 0,03 \cdot 1) = 110 \cdot 1,03 = 113,3$$

Faktor domácích kamenů

Je důležité rovnoměrně rozložit kameny na domácí straně hrací desky, za tímto účelem byl přidán jeden ze dvou bonusů – faktor domácích kamenů.

Nechť s je základní hodnotící skóre (např. počet bodů) pro danou pozici hráče. Definujme oblast domácí desky H jako:

$$H = \begin{cases} \{19, 20, 21, 22, 23, 24\}, & \text{pokud je hráč bílý} \\ \{1, 2, 3, 4, 5, 6\}, & \text{pokud je hráč černý} \end{cases}$$

Nechť h je počet kamenů hráče obsazujících tyto body:

$$h = \sum_{i \in H} c_i$$

kde c_i je počet kamenů, které hráč vlastní na bodu i .

Poté se upravené hodnotící skóre s' vypočítá jako:

$$s' = s \cdot (1 - \gamma \cdot h) \text{ kde } \gamma$$

$= 0,01$ je bonusový koeficient za každý kámen v domácí desce.

Pokud má hráč $s = 140$ a 4 kameny v domácí zóně, pak:

$$s' = 140 \cdot (1 - 0,01 \cdot 4) = 140 \cdot 0,96 = 134,4$$

Tento faktor odměňuje včasné soustředění kamenů do domácího pole, čímž urychluje přechod do fáze vyřazování. Tyto kameny blokují potenciální výstup kamenů soupeře z bar, což způsobuje zmeškaný tah.

Faktor prime

Jedním z nejdůležitějších faktorů, které se mohou ve hře vyskytnout, je blokování pozic pomocí prime. Prime je řada zablokovaných bodů, přes které se soupeř nemůže pohybovat. Nepřímo rozděluje desku na dvě části a omezuje schopnost soupeře se pohybovat.

Nechť s představuje základní hodnotu skóre (např. počet bodů) pro danou pozici hráče. Nechť je hrací plocha reprezentována posloupností bodů $P_1, P_2, \dots, P_{24}$ a nechť b_i je indikační funkce splňující následující podmínky:

$$b_i = \begin{cases} 1, & \text{pokud } P_i \text{ patří hráči a má } \geq 2 \text{ kameny} \\ 0, & \text{v opačném případě} \end{cases}$$

Nechť k označuje maximální počet po sobě jdoucích $b_i = 1$, tj. délku nejdelšího prvočísla. Pak je upravené skóre s' dáno vztahem:

$$s' = \begin{cases} s, & \text{pokud } k < 2 \\ s \cdot (1 - \delta \cdot (k - 1)), & \text{pokud } k \geq 2 \end{cases}$$

kde $\delta = 0,05$ je bonusový koeficient za bod v prvenství nad rámec prvního.

Předpokládejme, že hráč má $s = 120$ a sérii 4 po sobě jdoucích zablokovaných bodů. Pak:

$$s' = 120 \cdot (1 - 0,05 \cdot (4 - 1)) = 120 \cdot 0,85 = 102$$

Tento faktor motivuje AI k vytváření delších nepřerušovaných řad, které jsou klíčové pro uvěznění soupeřových kamenů a kontrolu území. Je poměrně obtížné je vytvořit, ale pokud již řada existuje, algoritmus se ji bude snažit zachovat. Funguje to dobře v kombinaci s bonusem za kameny na domácí desce.

6.3 Testy

Hodnotící funkce byla testována pomocí testů rozdělených do dvou kategorií:

1. Testy k určení relevance a důležitosti faktorů.
2. Testy proti lidským hráčům.

Před zahájením testování je důležité se ujistit, že počáteční bodovací funkce funguje pro každého hráče stejně. Bohužel není možné statisticky dokázat, že jsou dva hráči rovnocenní, ale je možné vyvrátit tvrzení, že tomu tak není.

Test zaujatosti hodnotící funkce

Aby se ověřilo, že základní hodnotící funkce neobsahuje žádné systémové zkreslení, byl proveden přímý srovnávací experiment, v němž proti sobě odehráli dva agenti, používající identickou základní hodnotící funkci, celkem $n = 200$ her. Výsledky byly následující:

- Hráč 1 vyhrál: 98
- Vítězství hráče 2: 102

Nulová hypotéza zní:

$$H_0 : p = 0,5$$

proti oboustranné alternativě:

$$H_1 : p \neq 0,5$$

p označuje skutečnou pravděpodobnost, že hráč 1 vyhraje hru. Jedná se o binomický test za předpokladu nezávislých her a absence poziční nebo implementační asymetrie.

Při použití binomického testu s parametry $n = 200$ a $x = 98$ je oboustranná hodnota $p = 0,8321$. Standardní prahová hodnota významnosti je $\alpha = 0,05$.

$$p = 0,8321 \gg \alpha = 0,05$$

Data podporují hypotézu, že základní hodnotící funkce je nezaujatá vůči straně hráče. Oba agenti vykazují statisticky rovnocenný výkon a hodnotící funkci lze považovat za symetrickou a platnou pro další kontrolované srovnání.

6.3.1 Relevance faktorů

Pro posouzení individuálních a kolektivních příspěvků různých hodnotících faktorů v AI pro backgammon byla provedena řada kontrolovaných experimentů. Každý test porovnával agenta používajícího jediný hodnotící faktor s referenčním agentem spoléhajícím se výhradně na základní hodnotící funkci založenou na bodech. Cílem bylo určit, zda některý z faktorů přináší statisticky významné zlepšení herního výkonu.

V každém experimentu odehráli oba agenti celkem 200 her a byl zaznamenán počet vítězství agenta vylepšeného daným faktorem. K výpočtu úrovně spolehlivosti, že upravený agent je silnější než základní, byl použit jednostranný binomický test. Tento statistický test přímo vyhodnocuje hypotézu, že daný faktor vede k lepšímu rozhodování a tím k většímu počtu vítězství.

Abychom zjistili, zda konkrétní hodnotící faktor významně zlepšuje výkon AI, modelujeme počet vítězství vylepšeného agenta jako binomickou náhodnou proměnnou. Necht' $n = 200$ je počet her odehraných v zápase a necht' x označuje počet vítězství dosažených agentem používajícím konkrétní faktor. V rámci nulové hypotézy, že oba agenti mají stejné schopnosti, předpokládáme, že pravděpodobnost výhry v jedné hře je:

$$H_0 : p = 0,5$$

Alternativní hypotéza je, že faktor zlepšuje výkon:

$$H_1 : p > 0,5$$

Jedná se o jednostranný test hypotézy. Binomický test vypočítá pravděpodobnost získání x nebo více vítězství n pokusů za předpokladu, že nulová hypotéza je pravdivá. Tato pravděpodobnost je **p-hodnota**:

$$\text{p-hodnota} = P(X \geq x \mid n, p = 0,5)$$

„Úroveň spolehlivosti“ se odvozuje jako:

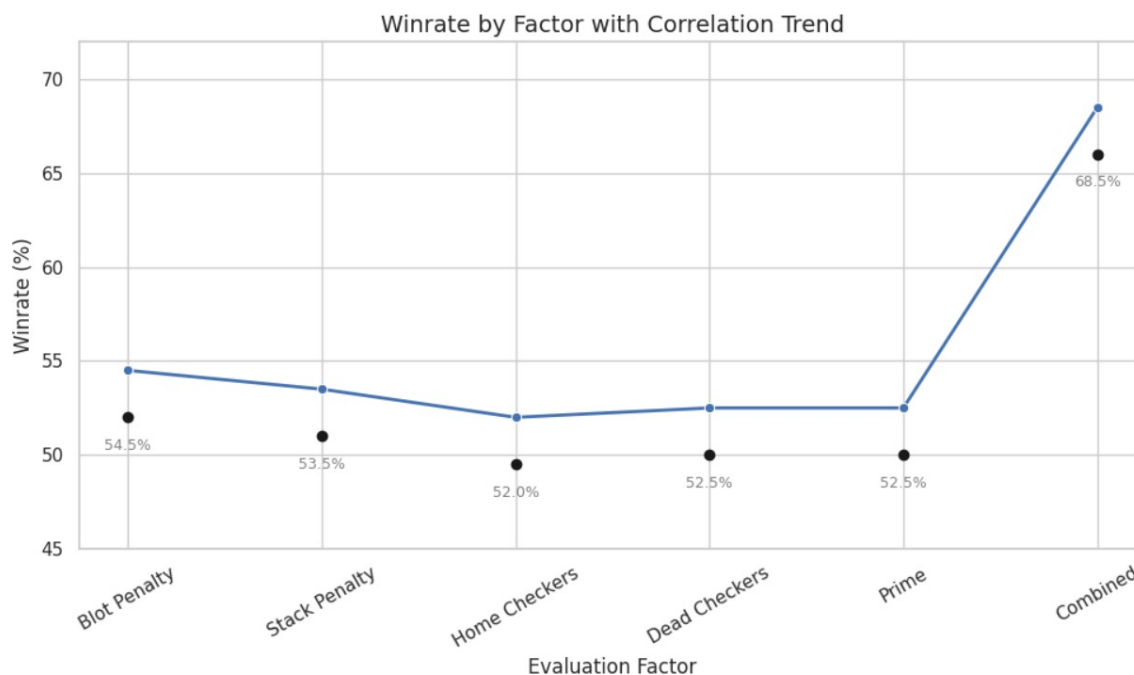
$$\text{Důvěra (\%)} = (1 - \text{p-hodnota}) \times 100$$

Tato hodnota udává statistickou sílu důkazu, že vylepšený agent je lepší než základní varianta. Úroveň spolehlivosti blížící se 100 % naznačuje, že je vysoce nepravděpodobné, že by pozorovaná úspěšnost byla výsledkem náhody za předpokladu stejné úrovně dovedností.

Výsledky jsou uvedeny v tabulce 6.1. Ačkoli žádný jednotlivý faktor nedosahuje konvenční prahové hodnoty významnosti 95 %, několik z nich se k ní blíží. Zejména faktory **penalizace za skvrny** a **penalizace za hromadění** vedou k míře výher nad 53 % s úrovní spolehlivosti přes 80 %. Nejdůležitějšího výsledku však dosáhneme, když zkombinujeme všechny faktory: agent vyhrává 137 z 200 her, čímž dosahuje míry výher 68,5 % a **úrovně spolehlivosti 100 %**. To potvrzuje, že faktory nejsou relevantní pouze jednotlivě, ale vykazují také synergii, což vede k výrazně silnější AI, jak je vidět na obrázku 6.3.

Faktor	Výhry	Výherní poměr	Spolehlivost (%)
Trest za blot	109	54,5 %	88,54
Trest za hromadění	107	53,5 %	82,10
Domácí šachy	104	52,0 %	68,96 %
Mrtvé dámy	105	52,5	73,77
Prime	105	52,5 %	73,77 %
Celkem	137	68,5 %	100,00

Tabulka 6.1: Výherní poměry a statistická spolehlivost pro různé hodnotící faktory (po 200 hrách)



Obrázek 6.3: Míra výher podle faktoru

Faktory fungují nejlépe společně, ale i jednotlivě mohou přinést malé zlepšení. Kombinace domácích kamenů a faktorů prime se ukázala jako nejlepší. I když jednotlivě výrazně nezlepšily výkon AI, společně vytvořily situace, kdy se prime vytvořil blízko domácí zóny hráče, což nejen zablokovalo

kamenným soupeře v opuštění bar, ale také zablokovala kamenné soupeře na prvních pozicích, což donutilo soupeře rozptýlit své kamenné, čímž vytvořil zranitelné zóny, nebo naopak, nechat příliš mnoho kamenů na jednom bodě. To vede k dominanci nad soupeřem, jak je vidět na obrázku 6.4.



Obrázek 6.4: Kombinace faktorů vs. základní hodnocení

Testování jednotlivých faktorů proti sobě nepřineslo žádné významné výsledky, výherní poměr odehraných her se pohybuje kolem 50 % a není možné posoudit, který faktor je silnější než ten druhý. Kombinace faktorů mohou být silnější, ale nebyly dostatečně otestovány, aby bylo možné s jistotou tvrdit jejich převahu. Nicméně lze identifikovat nejslibnější kombinace: Trest za blot + Trest za stack, Bonus za domácí kameny + Bonus za prime, Bonus za domácí kameny + Trest za mrtvé kameny.

Se všemi aktivovanými faktory se AI stává poměrně silnou. Pokud základní hodnocení vede k příliš agresivní hře, aktivované faktory jí umožňují najít velmi nenápadné tahy a dlouhodobě obrátit to, co lze nazvat „štěstím“, ve svůj prospěch. Faktory hrají rozhodující roli při výběru tahu, když se obvyklé skóre základní funkce pohybuje v rozmezí 100–150, s povolenými faktory, které mají kumulativní účinek, se toto skóre měří v řádu několika set a často přesahuje 1000, kde vzdálenost v bodech již nehraje významnou roli. Tyto faktory však mají významnou nevýhodu: nevyhodnocují šachovnici dynamicky. Jejich dopad na hru lze výrazně zvýšit úpravou modifikátorů bonusů nebo penalizací během hry.

6.3.2 Testy proti lidským hráčům

Jako doplněk k experimentům s automatickou hrou a statistickým experimentům byly provedeny přímé testy člověk versus AI, aby se vyhodnotilo, jak dobře si AI vede proti lidské intuici a strategickému

uvažování. Byly odehrány čtyři sady po 15 hrách. Dvě sady zahrnovaly **začínajícího** hráče, který se teprve nedávno naučil pravidla backgammonu. Další dvě sady odehrál **zkušený** hráč s praktickými zkušenostmi, ale bez profesionálního tréninku.

Každý lidský soupeř odehrál jednu sadu proti **základní hodnotící** AI (která používá pouze počet bodů) a jednu sadu proti **vylepšené hodnotící** AI (která zahrnuje všechny implementované faktory: penalizaci za blot, penalizaci za stack, domácí kameny, mrtvé kameny a formaci prime).

Výsledky jsou shrnuty v tabulce 6.2. Základní AI dokázala porazit začátečníka (9:6), ale byla výrazně poražena zkušeným hráčem (5:10), což naznačuje, že hodnocení založené pouze na počtu bodů nestačí k tomu, aby odolalo ani průměrné lidské strategii. Když však byly povoleny všechny faktory hodnocení, AI vykázala dramatické zlepšení. Dominovala nad začátečníkem (14:1) a, co je důležitější, zvrátila svou prohru proti zkušenému hráči tím, že vyhrála 11 z 15 her.

Tabulka 6.2: Výherní poměry AI proti lidským soupeřům (15 her v sadě)

Typ soupeře	Konfigurace AI	Výherní poměr (AI)
Začátečník	Základní hodnocení	60,0 % (9 z 15)
Začátečník	Všechny faktory povoleny	93,3 % (14 / 15)
Zkušený	Základní hodnocení	33,3 % (5 / 15)
Zkušený	Všechny faktory povoleny	73,3 % (11 / 15)

Přidání faktorů umožnilo AI rozpoznat a využít poziční slabiny, vyhnout se taktickým chybám a vyvíjet konzistentní tlak – vlastnosti.

Každý test v této sekci měl pro hráče AI hloubku prohledávání 3 tahy. Průměrná délka partie se značně lišila. Nejdelší partie se odehrály mezi základními hodnotícími funkcemi, kde se prostě navzájem útočily tak často, jak to bylo možné, a partie někdy trvala 90 nebo více tahů. Nejkratší partie se odehrály mezi začátečníkem a AI se všemi faktory zapnutými a mezi AI se základní funkcí a AI s faktory – partie trvaly asi 40 tahů, přičemž AI s faktory zcela dominovala. V průměru partie trvaly asi 60 tahů, což je relativně dlouhá doba.

Průměrná doba tahu se také lišila a přímo závisí na počtu dostupných tahů a počtu jader procesoru, která vypočítávají hodnotu tahů. Obvykle doba výpočtu nepřesahuje 8 sekund, průměrná hodnota je 5,6 sekundy se všemi faktory zapnutými pro 14jádrový procesor. Základní hodnotící funkce počítá stavy hry asi dvakrát rychleji než verze s faktory. Existují případy, kdy je několik set možných tahů, takže funkce potřebuje mnohem více času a výpočet může trvat déle než minutu. U procesorů s malým počtem jader se doba ještě více prodlužuje. Použití transpoziční tabulky učinilo algoritmus nenáročným z hlediska využití RAM. Za normálních okolností využití paměti nepřesahuje 600–700 MB. Pro situace, kdy je třeba vyhodnotit velký počet tahů, se toto číslo zvýšilo na 2,5 GB, což pro moderní počítače není velké množství. To umožňuje spustit hru i s prohledáváním až do 4 tahů dopředu. Současně však proces vyhodnocování počítačem trvá extrémně dlouho, v průměru několik minut na každý tah, takže toto použití se nedoporučuje.

Kapitola 7

Závěr

Cílem této práce bylo vyvinout kompletní implementaci hry Backgammon, zahrnující jak základní herní logiku, tak konkurenceschopnou AI. Je založena na algoritmu Expecti-MiniMax, který rozšiřuje klasický přístup MiniMax o zohlednění stochastických výsledků, jako jsou hody kostkami.

Pro vylepšení rozhodovacích schopností AI byla vytvořena modulární hodnotící funkce. Základní vrstva hodnocení využívá počet bodů – standardní měřítko pokroku v backgammonu – k odhadu žádoucnosti herního stavu. Navíc byla zavedena řada strategicky motivovaných hodnotících faktorů. Jedná se o sadu bonusů a penalizací navržených tak, aby skóre pozice bylo „spravedlivější“.

Testováním, a to jak v rámci hry proti sobě samému, tak proti lidským soupeřům, se ukázalo, že i když jednotlivé faktory přinášejí jen mírná zlepšení, jejich kombinovaný účinek vede k výrazně silnější AI. Vylepšená hodnotící funkce překonala základní model jak statisticky, tak v reálných hrách. Tyto výsledky potvrzují, že strategické faktory, jsou-li integrovány do strukturovaného hodnotícího modelu, mohou zvýšit výkon AI ve složitých a pravděpodobnostních hrách, jako je backgammon.

Byly prozkoumány a implementovány různé způsoby optimalizace algoritmu a stále existuje prostor pro další optimalizaci.

Závěrečné úvahy

Algoritmus se ukázal být dostatečně silný na to, aby porazil zkušeného hráče, ale zároveň předpokládám, že si neporadí s hráči, kteří dobře studovali teorii hry backgammon a především dokážou dynamicky měnit svou taktiku. Samotný algoritmus nedokáže přímo přepínat mezi agresivní a bezpečnou hrou. Nezohledňuje další faktory, které také mají vliv na hru, ale v tomto konkrétním případě se statisticky neprokázaly jako významné. Jedná se o kotvy, strategicky důležité body na hrací desce (ne všechny jsou stejné), oblasti pro vytváření útoků a tak dále. Takové faktory by přetížily hodnotící funkci, která je volána mnohokrát, takže jsem musel vybrat ty nejrelevantnější. Profesionální hráči o těchto částech strategie také uvažují a s prostředky nakládají moudřeji. Ve skutečnosti se algoritmus ExpectiMiniMax v současné době blíží svému vrcholu a další optimalizace z něj mohou vytvořit zcela nový algoritmus. Nalezené faktory jsou užitečné například pro použití v neuronových sítích, což je slibnější přístup. Nicméně implementovaná verze stačí k tomu, aby byla hra hratelná a zajímavá pro většinu lidí, přičemž složitost lze upravit přidáním různých faktorů a je možné hrát s různými jejich kombinacemi.

Bibliografie

- [1] BACKGAMMON GALAXY. *Jak hrát backgammon* online. 2025. 19. 4. 2025. Dostupné na: <https://www.backgammongalaxy.com/how-to-play-backgammon>.
- [2] BACKGAMMON GALORE. *Pravidla backgammonu* online. 2025. 19. 4. 2025. Dostupné na: <https://www.bkgm.com/rules.html>.
- [3] BRUNNER, F. Mastering the game of Go with deep neural networks and tree search (Silver 2016). *2016 Heidelberg University*, 2019.
- [4] BURO, M. Probcut: An effective selective extension of the α - β algorithm. *ICGA Journal*. SAGE Publications Sage UK: Londýn, Anglie, 1995, sv. 18, č. 2, s. 71–76.
- [5] GOOGLE DEEPMIND. *AlphaZero a MuZero* online. 2025. 26. 4. 2025. Dostupné na: <https://deepmind.google/research/breakthroughs/alphazero-and-muzero/>.
- [6] HAUKE, T.; BURO, M. a SCHAEFFER, J. *-MINIMAX performance in backgammon. In: *Proceedings of the 4th International Conference on Computers and Games*. Berlín, Heidelberg: Springer-Verlag, 2004, s. 51–66. CG'04. ISBN 3540324887. Dostupné na: https://doi.org/10.1007/11674399_4.
- [7] LAI, Y.-L.; CHANG, S. K. Y. a KO, T.-C. Aplikace genetického algoritmu a samoučení v počítačovém Othellu. In: 2006. Dostupné na: <https://api.semanticscholar.org/CorpusID:17057822>.
- [8] LAMANOSA, R. G.; LIM, K. C.; Dominić, I.; MANARANG, T.; SAGUM, R. A. et al. Vylepšení Expectimaxu prostřednictvím paralelního vyhledávání pro nedeterministické hry. *International Journal of Future Computer and Communication*, 2013, s. 466–470. Dostupné na: <https://api.semanticscholar.org/CorpusID:54208618>.
- [9] MAN, R. de. *Syzygy Endgame Tablebases* online. 2018. 29. 4. 2025. Dostupné na: <https://syzygy-tables.info/>.
- [10] MICROSOFT DOCS. *Jak používat třídy kódování znaků v .NET* online. 2023. 30. 4. 2025. Dostupné na: <https://learn.microsoft.com/en-us/Dotnet/standard/base-types/character-encoding>.
- [11] RUSSELL, S. a NORVIG, P. *Umělá inteligence: Moderní přístup*. 3. vyd. Pearson, 2010. ISBN 978-0136042594.
- [12] TESAURO, G. Učení s časovým rozdílem a TD-Gammon. *Commun. ACM*. New York, NY, USA: Association for Computing Machinery, březen 1995, sv. 38, č. 3, s. 58–68. ISSN 0001-0782. Dostupné na: <https://doi.org/10.1145/203330.203343>.

Příloha A

Uživatelská příručka

Systémové požadavky

- **Operační systém:** Windows 10 nebo novější (64bitová verze)
- **Místo na disku:** přibližně 150 MB
- **.NET Runtime:** Není vyžadován, pokud používáte předkompilovanou verzi

Soubory v balíčku

Rozbalený archiv by měl mít následující strukturu:

```
Backgammon/  
|- Backgammon.sln  
|- Core/  
|  |- (základní herní logika a AI)  
|- Backgammon_UI/  
|  |- (uživatelské rozhraní WPF)  
|- Hra/  
|  |- Backgammon.exe
```

Možnost 1: Spuštění předkompilované verze (není třeba kompilovat)

1. Rozbalte archiv do libovolné složky.
2. Přejděte do adresáře `Game/`.
3. Spusťte soubor `Backgammon.exe`

Poznámka: Tato verze je samostatná sestava, která obsahuje všechny potřebné nativní knihovny a runtime .NET.

Možnost 2: Kompilace ze zdrojového kódu

Chcete-li sestavit projekt ze zdrojového kódu (možnosti 2A nebo 2B), musíte mít nainstalovány sadu **.NET 8.0 SDK**.

Metoda A: Spuštění přímo ze zdrojového kódu

Spusťte následující příkaz z kořenového adresáře:

```
dotnet build
dotnet run --project Backgammon
```

Metoda B: Publikování pro distribuci

Pomocí této metody byl vygenerován soubor `Backgammon.exe`.

```
dotnet publish "Backgammon/Backgammon.csproj" ^
  -c Release ^
  -r win-x64 ^
  --self-contained true ^
  /p:PublishSingleFile=true ^
  /p:IncludeNativeLibrariesForSelfExtract=true ^
  /p:Publikovat zkrácenou verzi=false ^
  -o "publish"
```

Tím se vygeneruje jediný soubor `.exe`, který lze spustit na jakémkoli kompatibilním počítači se systémem Windows bez nainstalovaného prostředí .NET. Výstup se uloží do adresáře `publish/`.

Spuštění hry

Po spuštění hry (pozor, první spuštění může chvíli trvat) se otevřou dvě okna:

- **Terminálové okno**, které poskytuje informace o běhu procesu vyhodnocování AI během hry. Toto okno se aktualizuje v reálném čase podle toho, jak jsou tahy vyhodnocovány.
- **Hlavní okno hry**, které slouží jako grafické rozhraní pro hraní Backgammonu.

Zahájení nové hry

Ve výchozím nastavení se hra otevírá v předvoleném režimu **Hráč proti hráči (PvP)**. Chcete-li nakonfigurovat a spustit novou hru:

1. Vyberte požadovaný **herní režim**:
 - Hráč proti hráči (PvP)
 - Hráč proti AI
 - AI proti AI
2. Pokud jste vybrali režim *Hráč proti AI*, vyberte **stranu hráče** (bílá nebo černá).
3. Pokud vybraný herní režim není PvP, zadejte **hloubku prohledávání** v tazích:
4. Klikněte na tlačítko „**Nová hra**“, aby se hra spustila s vybranými nastaveními.

Provedení tahu

Hráč, který začíná, je na začátku každé nové hry vybrán náhodně. Na začátku tahu každého hráče je třeba provést následující kroky:

1. Klikněte na tlačítko „**Hodit kostkou**“.
 - Hodnoty obou kostek se zobrazí jako **Kostka 1** a **Kostka 2**.
 - Pro informaci se zobrazí také poslední hod soupeře.
2. Po hodu kostkami klikněte na bod, ve kterém se nachází jedna z vašich figurek. Tím vyberete výchozí bod a všechny povolené cílové body se zvýrazní **zeleně**.
3. Klikněte na jeden ze zeleně zvýrazněných cílových bodů a tah dokončete.
4. Chcete-li vyřadit kamen, klikněte na bod, na kterém se nachází kamen, který lze vyřadit. Pokud je vyřazení povoleno, zvýrazní se příslušná zóna. Kliknutím na zónu vyřazení akci dokončíte.

Konfigurace režimu AI vs AI

Při výběru herního režimu **AI vs AI** můžete volitelně vybrat jeden nebo více z pěti hodnotících faktorů pro každou instanci AI samostatně. Vybrané faktory budou po dobu trvání hry aplikovány na příslušného hráče AI.

Ovládání provádění AI

Během hry AI vs AI můžete simulaci ovládat pomocí následujících tlačítek:

- **Zastavit:** Pozastaví hru po dokončení aktuálního tahu.
- **Pokračovat:** Pokračuje ve hře od místa, kde byla pozastavena.

Příloha B

Očekávaný počet bodů získaných za tah

V backgammonu závisí počet bodů, o který se hráč může posunout v jednom tahu, na výsledku hodu dvěma šestistrannými kostkami. Každý tah se skládá buď z nezdvojeného hodu (dvě různá čísla), nebo ze zdvojeného hodu (obě kostky ukazují stejné číslo). Očekávaný počet posunutých bodů lze vypočítat zohledněním všech 36 možných výsledků hodu kostkami a výpočtem průměrné celkové vzdálenosti tahu na jeden tah.

Rozlišujeme dva případy:

- **Nedvojice:** Existuje 30 výsledků, kdy obě kostky ukazují různá čísla. Každý takový výsledek umožňuje hráči provést dva tahy, jeden s každou kostkou. Například hod 2 a 5 umožňuje hráči posunout jednu figurku o 2 body a druhou o 5 bodů, což dává celkem 7.
- **Dvojice:** Existuje 6 výsledků, kdy obě kostky ukazují stejné číslo (např. 3–3). V tomto případě může hráč provést čtyři tahy, každý o hodnotě dané kostky. Například 3–3 umožňuje čtyři tahy o 3, celkem 12 bodů.

Pro výpočet očekávaného počtu bodů, o které se hráč posune v jednom tahu v backgammonu, rozdělíme výpočet na dva odlišné případy: **nezdvojené hody** a **zdvojené hody**. Konečná očekávaná hodnota se vypočítá jako vážený průměr na základě jejich příslušných pravděpodobností.

Existuje 30 různých kombinací dvou kostek bez dvojice (např. (1,2), (1,3), ..., (5,6)). V takových případech se každá kostka použije jednou a celkový počet bodů získaných v tahu je součtem obou kostek.

Nechť $S_{\text{non-double}}$ označuje průměrný posun při hodu bez dvojice:

$$S_{\text{non-double}} = \frac{1}{15} \sum_{1 \leq i < j \leq 6} (i+j)$$

Výčet všech jedinečných párů a jejich součtů:

$$\text{Součty} = 3 + 4 + 5 + 6 + 7 + 5 + 6 + 7 + 8 + 7 + 8 + 9 + 9 + 10 + 11 = 105$$

Takových kombinací je 15, takže:

$$S_{\text{bez zdvojení}} = \frac{105}{15} = 7$$

Existuje 6 možných dvojitych hodů: (1,1), (2,2), ..., (6,6). Při dvojitěm hodu se každá hodnota kostky hraje čtyřikrát. Celkový počet získaných bodů je tedy $4 \cdot d$, kde d je hodnota kostky.

Nechť S_{double} je průměrný posun při dvojitěm hodu:

$$S_{\text{double}} = \frac{1}{6} \sum_{d=1}^6 4d = \frac{1}{6} \cdot 4 \cdot \sum_{d=1}^6 d = \frac{1}{6} \cdot 4 \cdot \frac{6 \cdot 7}{2} = \frac{84}{6} = 14$$

Pravděpodobnost, že nepadne dvojice, je $\frac{5}{6}$, a pravděpodobnost, že padne dvojice, je $\frac{1}{6}$. očekávaný počet bodů získaných za tah je:

$$E = \frac{5}{6} \cdot S_{\text{ne-dvojnásobek}} + \frac{1}{6} \cdot S_{\text{dvojnásobek}} = \frac{5}{6} \cdot 7 + \frac{1}{6} \cdot 14 = \frac{35}{6} + \frac{14}{6} = \frac{49}{6}$$

$$E = \frac{49}{6} \approx 8,17$$

Tato hodnota se používá v implementaci ExpectiMiniMax v AI jako základní odhad pozičního pokroku soupeře během vynechaných tahů.